Advancing Dataflow Compilation with MLIR: Multidimensional Ownership-based Scheduling and Extraction of Affine Loops in Synchronous Dataflow

Pedro Ciambra

February 2025

Advisors: Hervé Yviquel, Maxime Pelcat, Mickaël Dardaillon

Contents

1 Introduction **3**
1.1 Organization of the thesis 4

2 Background **5**
2.1 Dataflow 5
2.1.1 Dynamic Models of Computation 6
2.1.2 Static Models of Computation 6
2.2 Intermediate representations 7

3 Related works **10**

4 Problem Statement **11**
4.1 Multi-rate SDF scheduling 11
4.2 Granularity management 12
4.3 Task regularity and affine loops 12
4.4 Multidimensional dataflow 13
4.5 Memory management and the need for an ABI 15
4.6 Hypotheses 17

5 Methodology **18**
5.1 Target platform 18
5.2 Reference implementations 19
5.3 Applications 19
5.3.1 Cholesky factorization 19
5.3.2 Scale Invariant Feature Transform (SIFT) 19
5.4 Profiling and benchmarking 20
5.5 Development 20

6 Target contributions **21**
6.1 Multi-rate SDF scheduler and runtime 21
6.1.1 Memory allocation and ownership semantics in SDF 23
6.1.2 Heap allocation, delays and read-write ports 23
6.2 Affine loop lowering 25
6.3 IaRa 2.0 26
6.3.1 The `iara` MLIR dialect 27
6.3.2 Actor ABI 28
6.4 Layout-Aware Dataflow 28

7 Chronogram **29**

8 References **31**

9 Appendix **36**

Abstract

The Synchronous Dataflow (SDF) model of computation is a foundational tool for designing deterministic parallel systems, particularly in embedded and signal processing domains. However, traditional dataflow scheduling methods often struggle to effectively extract parallelism from applications with varying granularities and complex data layouts. Fine-grained scheduling, while precise, incurs significant compilation and runtime overheads, a challenge that other areas of the compilation ecosystem, such as affine and polyhedral optimization, have addressed more successfully. This thesis introduces an enhanced version of the IaRa dataflow compiler, incorporating extensions to the SDF model to support high-level multidimensional semantics, data ownership, and fine-grained control over memory layout. These extensions are integrated into the MLIR compiler ecosystem, leveraging its modular and extensible architecture. Furthermore, the thesis proposes a novel task-based scheduler for multi-rate SDF, avoiding the limitations of single-rate conversion while maintaining determinism and efficiency. The central hypothesis is that simplifying the conversion between dataflow and affine representations provides performance gains by leveraging highly optimized compiler pipelines designed for affine loops. This approach aims to bridge the gap between dataflow models and advanced compilation techniques, fostering improved performance and scalability.

1 Introduction

As a continuation of the author's MsC work [13], the main objective of this PhD thesis is to demonstrate the benefits of adopting modular and standardized tools, interfaces and protocols in the development of dataflow compilers. In particular, we focus on MLIR, a recent effort in generalizing and modularizing compilers and Intermediate Representations (IRs) to adapt to the increasing variety of the software and hardware landscapes.

MLIR has received considerable attention from various research fields, including High Performance Computing (HPC), embedded systems, high level synthesis (HLS) and others. By providing flexible, modular and composable mapping between hardware features and high-level abstractions, it allows researchers to avoid the implementation of unrelated groundwork and to focus directly on the relevant abstractions for the research question they wish to answer. This property is generally lacking in existing dataflow research, which relies on low-level code generation and tightly-integrated frameworks that are difficult to extend and reuse without deep refactoring effort.

We aim to demonstrate the complementarity of these two research fields through the development of the IaRa 2.0 compiler. It expands on its previous iteration through the addition of an improved, idiomatic MLIR dialect that encodes the semantics of dataflow. The dialect is built with extensibility in mind; support for new dataflow Models of Computation (MoCs) can be added without a fundamental change in the structure of the IR. In the same spirit, the effort for the implementation of different scheduling strategies is reduced in comparison to compilers with more restricted IRs, as well as their integration with other compilation techniques (including ones designed for other fields of compiler research).

Given the wide adoption of MLIR by many areas of research, combined with dataflow's own considerably varied ecosystem, the space of research opportunities is wide, with a large variety of MoCs, target architectures, reference applications and scheduling schemes to study. After a period of literature review, in which we have considered ideas such as high-level synthesis, automated design space exploration, profile-guided optimization, near-memory computing, and automated clustering/splitting, we have settled on the scope of **affine loop detection and lowering for multidimensional Synchronous Dataflow (SDF) graphs in the context of multi-core RAM architectures**. This choice is informed by the variety of existing high-quality procedural programming optimization techniques currently available for this target platform (an example of which is polyhedral compilation), and validated by the existence of works in the field of dataflow that go in a similar direction (listed in sections 2, 3 and 4).

Through the integration of techniques from both fields under the same compiler framework, we hope to demonstrate that the areas of dataflow and low-level procedural compilation have the potential to compliment each other. However, to achieve this, it must be possible to convert algorithms (or sections of algorithms) between these two models. There exists a semantic gap between dataflow and procedural programming; dataflow MoCs are target-agnostic and do not impose a specific memory scheme, delegating memory management and representation to the scheduler; at the same time, low-level procedural programming is able to reference memory without constraint, and memory management is left as the programmer's responsibility. Memory management, however, is its own field of research, and relegating it to the role of implementation detail hides a significant amount of complexity and - importantly - technical and scientific knowledge. Dataflow compilers that target multi-core RAM architectures fill this semantic gap through the im-

plementation of bespoke schedulers, and the choice of memory management algorithm, if described at all in publication, is not often developed with interfaces that allow extension, replacement or interoperation with other tools.

To address this, we provide our own set of memory-aware compiler conventions, designed to follow the practice and semantics of already existing works in the compiler ecosystem, particularly where it comes to the representation of multidimensional data.

1.1 Organization of the thesis

Even with the aforementioned narrowing of scope, we still run into the fundamental issue that compilers are complex and multifaceted. The development of a new compiler requires multiple significant design decisions, many of which may have research merit individually, but which must be implemented concurrently, as their functioning depends on each other. Validating these decisions independently through experimentation is non-trivial. Comparison with other compilers must take into account the entirety of each compiler, which makes it difficult to attribute gains or losses to a specific component. At the same time comparison between different strategies inside a single compiler requires the reimplementation of reference algorithms from the state-of-the-art. In comparison to research projects that focus on the extension of existing compilers with new algorithms or techniques, the amount of groundwork required for our new compiler is non-trivial.

That being said, an effort is made in separating the components of the compiler into four discrete contributions. They consist of:

1. A runtime for SDF that uses compile-time knowledge to optimize dynamic memory allocation;
2. A pipeline for the detection and lowering of affine computations in SDF graphs, as well as delegating their compilation and optimization to specialized tools such as polyhedral compilers;
3. The architecture of the IaRa compiler itself, including the MLIR dialect and a proposal for standardized C ABI for the interoperability of actor implementations;
4. A proposal for a new MoC that extends SDF with multidimensionality and awareness of memory ownership, in a way that preserves compatibility with the current ecosystem of multidimensional compilers.

The remainder document is divided into:

- Section 2: Relevant background on the fields of dataflow and procedural language compilers;
- Section 3: An overview of related works that approach similar issues;
- Section 4: A problem statement that goes over the limitations of existing solutions;
- Section 5: A description of our chosen research methodology;
- Section 6: Our proposals, separated into the four discrete contributions;
- Section 7: A chronogram detailing the completed and projected PhD activities;

- Section 8: The bibliography;
- and Section 9: an appendix containing an unpublished paper describing contribution 4 in detail.

2 Background

In this section, we briefly introduce the research field of dataflow, as well as the current state of the art in compilers and intermediate representations. As both research fields are extensive, we omit details that do not directly relate to the contributions of this thesis.

2.1 Dataflow

Dataflow is a family of MoCs that provides an abstract representation of parallel programs, sharing similarities with paradigms such as *stream processing* and *reactive programming*. Its independence from specific parallelization mechanisms allows for broad applicability, with applications in areas like hardware design and machine learning. The concept of representing data transformations as flow graphs has a long history, emerging in hardware research during the 1970s and 1980s and appearing as a programming paradigm as early as 1960 [39]. Additionally, due to its compatibility with graphical representation, dataflow has been a subject of interest in visual programming research [72].

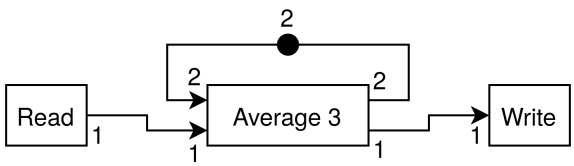

Figure 1: An example dataflow graph. Specifically, this is an SDF graph implementing a low-pass filter. The numbers denote the amount of tokens consumed/produced by each edge at each firing of an actor, and the black circle represents a delay of 2 initial tokens.

Dataflow models represent algorithms as directed graphs, as illustrated in Fig. 1. In these graphs, nodes (referred to as "actors") correspond to processing steps, while edges represent logical queues (first-in-first-out data structures, or FIFOs) that store data between processing steps. Each actor encapsulates an operation that can be executed (or "fired") to perform work; as it fires, the actor consumes values (referred to as "tokens") from its input edges and produces tokens onto its output edges. These firing actions are treated as indivisible units of work and are typically modeled as black-box operations. By scheduling the firing of actors in an appropriate order, data undergoes transformations as it moves from edge to edge, ultimately reaching the sink nodes as the algorithm's final output. This representation captures the data dependencies between different parts of the algorithm, simplifying the identification of parallelizable regions. This task can be automated, either dynamically by a runtime system or statically during compilation.

Dataflow is an architecture-agnostic representation and does not encode memory information; as long as data dependencies are respected, there is no expectation that tokens be stored contiguously in memory. Because this allows pipelining and streaming, dataflow

can be used in memory-constrained platforms such as Digital Signal Processors (DSPs) and Field Programmable Gate Arrays (FPGAs).

Dataflow systems can be implemented at varying levels of restrictiveness, trading off design flexibility with statically-known information for compile-time analysis and optimization. Many dataflow MoCs have been defined over the years [64]; due to space constraints, only a subset is covered in this report.

2.1.1 Dynamic Models of Computation

The most flexible MoCs are capable of dynamically adapting the behavior of their actors to the data contained within the tokens. While this increases programmer and algorithm flexibility, it also limits the ability to generate optimized schedules in advance; as a result, such systems depend on robust runtime mechanisms for efficient scheduling and memory management. One example of dynamic MoCs, Dataflow Process Network (DPN), has the property of allowing non-deterministic algorithms; this is useful for situations that deal with unpredictability, such as resource management problems like load balancing. Kahn Process Network (KPN) is a more constrained MoC where the relative order of actor execution is always the same for a given input, providing a level of predictability that enables static analysis and optimizations, as well as being more suitable for hardware synthesis. These MoCs have been described by Lee and Parks in 1995 [42], and remain relevant in research today [60].

2.1.2 Static Models of Computation

By introducing additional constraints, we arrive at MoCs with entirely data-independent behavior. This means that execution and memory allocation can be fully scheduled in advance. Although such models are not suitable for applications that depend on data-encoded behavior, they enable fine-grained optimizations that are not feasible with dynamic MoCs.

Synchronous Dataflow (SDF) represents the most constrained of the MoCs. By requiring that the number of tokens consumed and produced by an actor at each port remains constant across every activation, the system's behavior becomes predictable, and an efficient schedule can be determined at compile-time. This approach eliminates the need for overhead-intensive dynamic runtimes, making it particularly advantageous for resource-constrained or real-time applications.

A further constraint to SDF is Single-Rate SDF, where all edges have matching input and output rates; this means that all actors fire at the same rate [8]. Any SDF graph can be converted into a single-rate graph by duplicating edges and inserting fork/join actors to distribute the token flow between the copies; because this transformation simplifies the memory and execution dependencies of the actor firings, it is a common strategy as an intermediary step for SDF schedulers [55]. An example of this is explained in section 4.1 and Fig. 2. However, this approach has drawbacks, covered in section 4.

Figure 1 illustrates a simple example of an SDF graph implementing a smoothing filter. During each firing, the *Average 3* actor computes the mean of the three input values, sends the result to the output node, and sends copies of the two most recently read values back into its feedback port for use in the next firing. These values are stored in the FIFO queue associated with the self-loop, which must have capacity for storing at least two values. FIFOs that form loops require initialization with specific values (referred

to as "delays") to ensure that dependent nodes can execute during the first firing.

Not every configuration of rates leads to an admissible graph (that is, a graph that doesn't lead to input starvation or deadlocks).

The generation of a correct maximum-throughput multi-core schedule for dataflow graphs is an exponentially complex problem [27,71]; added to the fact that the latency of actor execution may be inconsistent and platform-dependent, this has led most dataflow compiler implementations to rely on heuristics and best-effort scheduler strategies.

The **Parameterized/Interfaced Dataflow Meta Model (PiMM)** [21] extends Dataflow MoCs with features that aim to improve code reuse and modularity. *Parameters* are statically-known values in terms of which the rates of actor ports can be defined, allowing the construction of algorithms that are generic to compile-time-known constants (image sizes, for instance). *Interfaces* consist of the definition of input and output ports for a graph, allowing them to be used as actors themselves; this gives the programmer the ability to define algorithms hierarchically and to reduce repetition.

Applying PiMM to SDF produces the **piSDF** (or πSDF) MoC. To allow the analysis of graphs defined with these features, they are typically flattened into a standard SDF graph as part of the scheduling process. This is analogous to template instantiation, constant propagation and function inlining in procedural language compilers.

The canonical implementation of piSDF, Spider (section 3), employs parameters to achieve reconfigurability. Instead of treating parameters as perpetually constant values, the runtime is able to adapt to sporadic changes, generating a new schedule as needed. This is useful for use cases with variable-sized inputs, such as image processing and compression algorithms.

Multidimensional Synchronous Dataflow (MDSDF) [51], **ArrayOL** [11, 26] and **Windowed Synchronous Dataflow (WSDF)** [38] are high-level MoCs for multidimensional signal processing applications. They address the issues stemming from representing multidimensional data through the single-dimensional semantics of previous dataflow MoCs (section 4.4) by replacing FIFOs with multidimensional grids, as well as providing the semantics for indexing subsets of these grids through matrix transformations.

In contribution 6.4, we propose a similar MoC that fixes its multidimensional semantics to those of MLIR, ensuring compatibility with existing tools and preserving similar capabilities.

Polyhedral Process Networks (PPN) [78] are a MoC that allows the detection and scheduling of dataflow topologies that conform to polyhedral form. While this work covers some common ground with ours, it benefits the most from targeting fine-grained graphs, optimizing and avoiding the synchronization cost of actors and showing a method for detecting polyhedral regions directly from the graph topology. Our approach is different; we assume that, for regular-task applications, most of the fine-grained parallelism can be easily extracted from multi-rate nodes and converted into affine loops, whose optimization is to be delegated to external MLIR-compatible tool; furthermore, using a polyhedral compiler such as Polygeist [50] allows us to optimize not only the dataflow topology, but also the implementation of the actors, in an integrated manner.

2.2 Intermediate representations

The challenge of optimizing code for parallel execution has been addressed numerous times and from various perspectives over the years. Projects such as OpenMP [17] en-

able the generation of parallel applications by annotating procedural code with special markers that identify parallelizable regions. Other projects have introduced their own approaches, such as Go's built-in coprocedures [57] and Rust's safety-oriented thread management [48]. A similar diversity exists in the domain of hardware accelerators, with frameworks like Nvidia CUDA [52] for general-purpose computing on GPUs being followed by AMD ROCm [2], Apple Metal [4], and the Khronos Group's OpenCL standard [70]. It is to be expected that many of these projects share common solutions that have been reimplemented across different platforms, with varying degrees of interoperability. This creates a significant challenge, as the lack of standardized interfaces and abstractions hinder the process of porting code between different programming languages and hardware platforms.

The solution for the problem of portability is standardized IRs. By focusing on generating a lower-level version of the algorithm in a widely adopted format, high-level compilers can automatically support any hardware platform, as long as a low-level compiler that supports this format has been developed for it. Symmetrically, new hardware platforms benefit from supporting the format, as it increases the number of high-level languages compatible with them.

A traditional choice of IR for high-level compilers is the **C language**. Since there has been wide platform support for a long time, new high-level MoC compilers need only provide translations from their own abstractions into C's MoC. Another benefit is the straightforward reuse of the immense quantity of existing C libraries. This strategy, referred to as a source-to-source compiler or "transpiler", is still very common, being adopted by both general-purpose programming languages and domain-specific languages (DSLs), including dataflow compilers such as Preesm [55].

Using textual C language as an intermediate representation has drawbacks. Despite its widespread support, the fact that it was designed to be written by humans introduces redundant steps to the integrated compilation pipeline, such as parsing the generated source code back into structured data. The language also lags behind the state-of-the-art where it comes to platform architecture; support for important and widely-available hardware features such as atomic operations and single instruction, multiple data (SIMD) directives were added to the C standard many years after they were already widespread in commercial hardware. To address this, the **LLVM IR** [41] was created; while it provides a human-readable form, it is primarily designed as a binary format to be efficiently machine-readable by low-level compilers. It has a level of abstraction between C and the assembly languages of common processor architectures: it foregoes C's high level features such as structured control flow and hierarchical symbol tables, and exposes low-level compilation abstractions such as dominance and hardware intrinsics.

While LLVM IR has become the industry standard for high-performance compilers, it, like C, covers a relatively narrow band of levels of abstraction. While many languages leverage its toolset extensively to avoid reimplementing low-level components of the compiler pipeline, a significant amount of higher-level compiler logic (such as type systems, lifetime analysis, macros, generics, meta-programming and early optimizations) is reimplemented from scratch by each project. This redundancy persists despite the similarities in functionality across different languages. For instance, the rise of machine learning has created a demand for flexible compilers capable of targeting a wide range of hardware accelerators, a variety of architectures for which LLVM IR was not originally designed. This has led to the emergence of several competing IR standards tailored for hardware-accelerated machine learning, such as TensorFlow [1], GLOW [63], and ONNX [6], all of

which have very similar features and semantics. Furthermore, each hardware vendor typically provides its own parallel programming interface, further fragmenting the ecosystem. Taking dataflow as another example, the limited memory management semantics of C and LLVM IR means that the high-level dataflow compiler must make a great deal of assumptions about the implementation of the actors, which sometimes remain undocumented. We expand on this topic in section 4.5.

To address these challenges, the **MLIR** [40] project was developed. It enables the integration of arbitrarily-defined domain-specific IRs within a unified environment, incorporating best practices in compiler design derived from the LLVM team's experience. Initially part of the TensorFlow project, MLIR was recognized for its broad utility and subsequently integrated into the LLVM project.

MLIR improves upon LLVM IR on several fronts. Among other features, it provides a common data structure for the simultaneous representation of different abstractions together in the same IR, allowing data from high-level compiler passes to be reused in low-level optimizations; it allows a hierarchical code topology, which helps modularize code transformations; and it has built-in semantics for high-performance multidimensionality, with tensors, affine operations, and hardware-aware memory references.

In MLIR, related compiler abstractions are organized into namespaces called *dialects*. Compiler developers can choose which dialects are relevant and permitted to interact, and they can also define and share their own dialects through the same unified semantics. MLIR includes built-in dialects for control flow, memory management, arithmetic and tensor operations, function and scope management, parallelization, and other concepts essential for general-purpose compilers. Backwards compatibility is also addressed: one of the dialects directly maps to LLVM IR and is frequently used as a target by projects for which the existing LLVM infrastructure remains relevant. This is our approach as well (section 6.3).

MLIR code can retain and propagate abstract data as needed, even deep into the compilation pipeline. Consider, for example, the common task of transforming an abstract matrix multiplication into executable code, either by converting it into a nested loop for CPU execution or by offloading it to an accelerator. Traditional compilation pipelines offer only two options: either annotate the code in advance with vendor-specific syntax or function calls, requiring developers to decide early how to compute the multiplication and limiting portability; or write the nested loop conventionally and rely on pattern-matching tools later in the compilation process to identify it as a matrix multiplication and replace it with an accelerator call. Since MLIR allows the operation to remain abstract even as the surrounding code is lowered, this decision can be deferred to a later stage in the compilation pipeline.

This approach improves the separation of concerns, as the hardware-specific optimization of matrix multiplication can be delegated to individual vendors, while the high-level compiler can focus on hardware-independent tasks. The tools used for transforming and manipulating source code and dialects remain consistent across all stages of compilation, enabling the development of high-quality, maintainable, composable and extensible compilers. By freeing users from having to choose between vendor lock or premature lowering, this approach also encourages the use of a more expressive, generic and declarative programming style. This capability is especially beneficial for domain-specific languages, as each domain can define its own semantics and delegate the implementation of other concepts to existing dialects, while retaining convenient access to them if needed. This is the approach taken by the dead code elimination pass developed during this author's MsC

dissertation [14].

3 Related works

Designing a compiler is a multifaceted problem. This is a non-exhaustive list of state-of-the-art works that each, individually, tackle some of the same challenges as we do.

Preesm [55] is a dataflow framework geared for rapid prototyping of DSP programs. It includes an Eclipse-based graphical interface and optimizing compiler that generates executables starting from topological descriptions of the dataflow graph and the target architecture, as well as user-defined piSDF actor implementations, provided in C/C++. It is bundled with Spider [32], a runtime for the piSDF MoC that is intended for the development of parallel programs on DSP processors, being presented as a direct alternative to OpenMP.

Preesm relies on C generation (section 2.2) as its choice of IR, and on conversion to single-rate dataflow (sections 2.1.2 and 4.1) It is this project's primary frame of reference and point of contrast to our MLIR-based approach (sections 2.2, 6.3).

Rust [48] is a general-purpose programming language whose focus on ownership-based semantics and safety features have led to a sharp rise in popularity in recent years, being now the first language other than C to be accepted into the Linux kernel. It leverages a robust type system to enforce ownership rules at compile time, with the goal of preventing the type of memory safety error that are pervasive in languages with unrestricted memory aliasing, such as C and C++. We borrow the concept of ownership and lifetime semantics into our dataflow scheduler and MoC.

OpenMP [17] is a widely-adopted cross-platform shared-memory parallel programming API that combines compiler directives, runtime library routines, and environment variables to enable incremental parallelization. Its fork-join execution model allows dynamic thread management through parallel regions, work-sharing constructs (e.g., `for`, `sections`), and task-based parallelism through a dynamic task-stealing scheduler. The runtime employs atomic operations and implicit barrier synchronization while supporting nested parallelism and heterogeneous offloading through device directives. Recent versions integrate task dependency tracking and cancellation primitives, making it suitable for both regular loop parallelism and irregular graph-based workloads. The task-based runtime models a static Directed Acyclic Graph (DAG), which is a subset of SDF; as such, it is an important point of comparison when measuring the effectiveness of dynamic schedulers and runtimes. **TaskGraph** [79] is an optimization for OpenMP's tasking model that makes use of Task Dependency Graphs. They work by using instrumentation to record a valid static schedule for a given DAG, and then using it to replace the costly synchronization overhead of the task-stealing runtime. While it is only applicable to deterministic DAGs, it has demonstrated that reducing contention is an effective strategy to achieve speedup in multicore scheduling. We measure our implementation of the Cholesky factorization algorithm against theirs (section 5.3.1).

Polygeist [50] is an advanced C compiler that specializes in polyhedral optimizations. It features a robust parser of the C language, and, as part of its front-end, converts it into standard MLIR with little loss of information. MLIR allows us to use it as a front-end to the C language in order to inspect the contents of actors and perform optimizations that cross the dataflow graph/kernel boundary.

4 Problem Statement

In the context of shared-memory multiprocessor architectures, a dataflow scheduler fulfills the function of translating a dataflow graph into procedural multi-threaded code. This requires the management of resources such as runtime memory, binary size, synchronization overhead, thread allocation, compilation time and programming effort; each of these tasks can be itself deeply complex. Different MoCs and target architectures provide different restrictions and requirements to this resource management, which often manifests as trade-offs when designing a scheduler. While the management of each of these resources is a complex problem in itself, they are similar in that they tend to be generally proportional to the size of the graph. In this report, we define them collectively as *scheduling pressure*, in analogy to the concept of *register pressure* in the context of traditional compilers.

Mitigating different forms of scheduling pressure is a focus of ongoing dataflow research, with strategies that range from MoC-specific memory allocation schemes [22], lower-level actor specification [20], and integrating techniques from other research fields, such as polyhedral compilation [12, 45].

In this section, we list some of the challenges involved in the development of SDF compilers. In Section 6, we address these challenges in the form of discrete contributions, as features of the IaRa compiler.

4.1 Multi-rate SDF scheduling

In comparison to single-rate SDF, multi-rate edges increase the complexity of multicore SDF schedulers. There may be many-to-many dependencies between individual firings of neighboring actors, whose amount, distribution and memory requirements do not necessarily follow patterns that can be trivially translated into a correct schedule. Because of this, a common strategy [22] is to convert the source multi-rate SDF into single-rate form. While this can be done for any SDF graph [19], as shown in Fig. 2, this may result in large effective graphs.

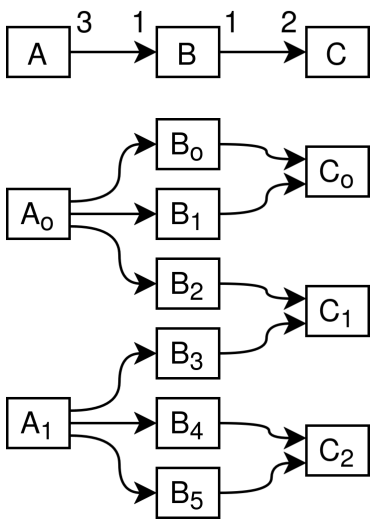

Figure 2: Conversion from multi-rate SDF into single-rate. The number of nodes grows based on the lowest common multiple of the rates.

This translates to increased scheduling pressure: a runtime scheduler will incur additional synchronization overhead, and an ahead-of-time (AoT) scheduler will require

additional static memory to store the expanded graph. Remedying this is a continuing focus of research, with surfacing strategies such as compact graph representation [5] and actor clustering [62, 76].

At the same time, this transformation dilutes information relating to the similarity of these tasks, which could otherwise be useful further down the analysis and optimization pipeline. To address the drawbacks of conversion to single-rate SDF, an integrated runtime scheduler that makes use of the compile-time information of SDF was developed, and presented in Section 6.1.

4.2 Granularity management

When scheduling a dataflow algorithm, a relevant property is the granularity of the graph being scheduled, a measurement of how evenly the total workload of the algorithm is divided between different actors. A coarse-grained graph has few actors that execute heavier workloads but have limited exposed parallelism, while a fine-grained graph exposes more parallelism through larger and more complex graphs, and actors that individually do a small amount of work. To ensure a balance between core utilization and scheduling pressure, a common design practice is to explicitly design algorithms for a target architecture, which limits the flexibility of the implementation and requires knowledge of the target architecture; additionally, optimizing schedulers may automatically increase the grain of the algorithm through transformations such as actor clustering [61]. The opposite operation (splitting actors into components) is more complex, as it requires a compiler to leave the bounds of dataflow by understanding the source code that the actors were programmed in, often in a lower level of abstraction. This has not yet been explored academically.

Dataflow compilers such as CAL [24] and LUSTRE [28] provide custom high-level DSLs for specifying both the graph topology and the implementation of the actors. While this affords a great deal of control to the compiler designer, this approach is limited when it comes to connecting external tools and keeping up with the state-of-the-art. In section 6.3, we show how IaRa leverages MLIR to address this limitation.

It is important to note that the choice of granularity is a problem in all parallel programming paradigms, and is highly dependent on the target architecture (not only the number of cores, but also memory and throughput/latency of inter-core communication). Research fields such as polyhedral compilation approach this through "auto-tuning", which consists of automated design space exploration and profile-guided optimization (PGO) [35, 36].

Our approach of converting SDF parallelism into affine form, described in section 6.2, and the use of MLIR 6.3 paves the way for the integration of these tools directly into the dataflow compilation pipeline.

4.3 Task regularity and affine loops

Task regularity, also referred in academia as task homogeneity, execution symmetry and execution pattern uniformity, is a property of parallel algorithms that relates to the similarity of the different tasks being executed at each thread [23]. For instance, image processing algorithms are often highly regular, as each pixel is processed in a similar way. This information is useful for scheduling and optimization, as this allows for accurate assumptions about the latency and memory behavior of the actor.

Conversely, irregular parallel algorithms simultaneously run different tasks on different

threads. Since predicting the runtime of a function ahead of time is analogous to the halting problem, this poses a problem for scheduling. Static schedulers that assign actor firings to specific threads ahead of time must rely on instrumentation and profiling to obtain this timing information, while schedulers must trade off complexity and overhead as they dynamically assign tasks to available threads and synchronize them as they are completed. As with granularity, this is a complex problem in most parallel programming models [68].

Often, the symmetry of homogeneous algorithms are a consequence of their reliance on affine loops. Affine loops are regions of procedural code with well-defined repeating operations whose dependencies can be derived by applying linear transformations over a constant range of values. In procedural programming languages, they are typically implemented with nested `for` loops with constant bounds. Many common applications can be described as affine loops, such as matrix operations and many image and signal processing algorithms.

When applicable, affine loops have a number of advantages over classical dataflow scheduling. As the dominant programming model for high performance computing, their optimization has continuously been a focus in compiler and hardware architecture development, having reached and sometimes surpassed what is achievable by manual programming [29]. Furthermore, it has been shown that the data symmetries that they expose automatically lead to speedups, in comparison to unstructured bulk parallelization [58]. The drawback is that they are less flexible when it comes to memory management, as memory addresses must also be derivable from linear transformations. They also typically do not handle memory allocation and deallocation; the availability of memory does not play a role in scheduling.

Given the advantages of affine loop scheduling, it is often more practical than dataflow when it comes to highly uniform parallel algorithms. A common strategy to leverage the fine-grained affine loop optimizations of procedural language compilers is to insert affine loops directly into actor implementations, either through the use of established libraries such as BLAS [9], or by using advanced affine optimizations on the actor themselves [45]. This is the strategy currently applied by most neural network frameworks, being effective at taking advantage of the modularity and automatic memory management of dataflow for the coarse grain and remaining performant at the fine grain [1, 54]. However, the fact that the two scheduling strategies are largely independent of each other means that inter-actor optimization is less feasible, as the details of the fine-grained structure is hidden from the dataflow scheduler.

Attempts to extract affine information directly from the dataflow topology have been successful [10, 77], but they remain coupled to their choice of framework. IaRa aims to provide a framework that simplifies experimentation with different optimization passes through the use of MLIR, which already provides native representation of affine loops and connects to several existing parallelization frameworks 6.2.

4.4 Multidimensional dataflow

Classical dataflow MoCs represent the transfer of data between actors with FIFO semantics; nodes consume tokens from the fronts of input queues and append to the backs of output queues. While this achieves some flexibility when generating a schedule and enables implicit data parallelism, this has the limitation of being strictly one-dimensional and only provides a single way to distribute data between tasks; unlike affine loops, they

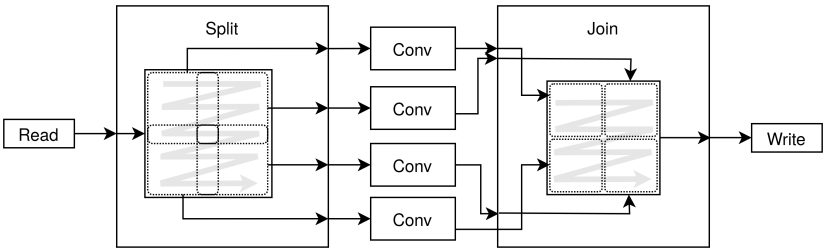

Figure 3: Parallelization of a 2D convolution. Four overlapping sections of the data must be processed by different cores and the results must be joined together into a contiguous image. Memory contiguity is represented by the gray arrows.

do not provide a natural way to map code execution to memory. Any special addressing would require a new actor for data reshaping, which adds otherwise unnecessary copies and scheduler pressure.

Conversely, in multidimensional algorithms, generation of optimized schedules often requires splitting buffers into arbitrarily sized components and along any of the data dimensions, which may result in complex memory access layouts. These partial references can be copied or modified and eventually recombined into the original buffer. A buffer may also have many overlapping references being addressed simultaneously, and the lifetime of its value must be managed to ensure correct data dependency.

Consider, for instance, the parallelization of a 2D convolution, as shown in Fig. 3. Because each operation needs access to neighbors of the element, the memory accessed by different cores may overlap. Correct execution depends on different regions of memory being available to each core at different times. In procedural programming, this must be done manually.

Some dataflow algorithm implementations work around this issue by dealing with references as data tokens themselves, modeling *for* loops in the language of dataflow [33]. While functional, this approach has the drawback of decoupling memory management from the dataflow model; while "iterator" actors avoid copies, they must be manually defined and the memory dependencies must be guaranteed by the programmer.

While multidimensional references are something that has become common in modern general-purpose programming languages, not many works in dataflow explore this type of representation. The original MDSDF [51] does not model memory, dealing directly with abstract tensors. ArrayOL [11] and WSDF [38] models overlapping execution by projecting the data into a sliding window layout, a manual, non-trivial fine-grained transformation.

Despite being intrinsically multidimensional, tensor-based neural network frameworks typically delegate the fine-grained parallelism to the underlying implementation of actors, relying on application parallelism and accepting the cost of increased memory usage. With the popularization of large AI models and the exploration of model scaling, this poses a problem, as memory requirements have grown substantially.

In order to allow multidimensional scheduling that allows for coarse-grained multidimensional slicing with automated memory management, we propose Layout-Aware Dataflow (LADF), a novel MoC that natively supports memory layout operations and implements ownership semantics 6.4.

4.5 Memory management and the need for an ABI

Dataflow MoCs have value semantics, which means that they only describe the dependencies and output of the program, leaving memory management to the implementation of the scheduler. This is appropriate as a high-level, platform-independent model, because dataflow can be applied in contexts other than multi-core random-access memory (RAM) architectures, such as in hardware synthesis and DSP platforms. However, in the context of multi-core RAM architectures, in practice, most high-performance kernels are implemented in the C language. Because C has reference semantics and tokens are referred to by their memory address, there is a significant semantic gap in the algorithm specification that ends up being defined by the implementation of the compiler. For instance, it is necessary to define a mapping between the actor signature and the C function interface of the kernel; before implementing the actors, it is necessary to know whether the memory accesses are expected to be read-write or read-only, if the memory of incoming edges can be reused, and whether the actor is responsible for allocating/deallocating the memory of tokens it produces/consumes. Collectively, these definitions form an application binary interface (ABI) (a collection of conventions that allow compiled programs to communicate [69]; in this case, the communication between scheduler and actor implementation). Because different dataflow compilers define different ABI, actors written for one dataflow compiler may not be immediately compatible with other compilers. An under-defined interface also has the effect of hiding information about potential optimizations from the scheduler, such as copy elision [34] and memory reuse. We tackle this problem by providing a well-defined calling convention 6.3.2 that covers the requirements of multidimensional dataflow and SDF.

As a case study, consider an SDF scheduler that relies on heap allocation, allocating memory immediately before the production of the value and deallocating it immediately after its consumption, as shown in Fig. 4.

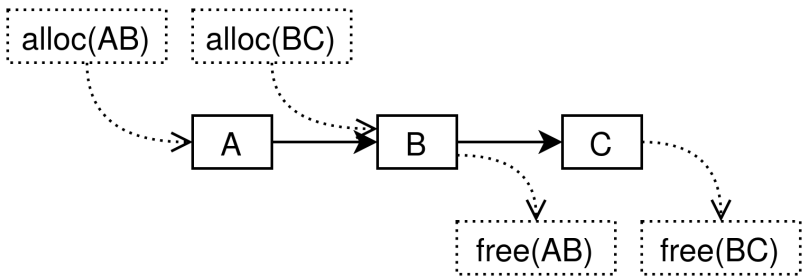

Figure 4: Memory management tasks in a single-rate SDF graph generated by an SDF scheduler; dotted elements were inserted automatically by the scheduler. FIFO queues are reduced to buffers named after there producer and consumer. AB buffer must be allocated before A can write into it, and can only be deallocated after B has read from it.

Since traditional dataflow MoCs do not encode memory information , memory management actors such as Fork and Join may cause spurious memory operations. Fig. 5 shows a possible transformation from a naïve heap allocation based scheduler that ties memory ownership to each token; the allocation and deallocation operations get multiplied together with their respective actors during conversion to single-rate. While there exist situations where a graph with this many dynamic memory management operations may still be useful (particularly with more distributed memory architectures, such as

distributed systems, in-memory processing platforms and GPUs), in the context of multi-core RAM architectures the high synchronization cost of dynamic memory allocations, deallocations and copies makes this solution unsuitable for fine-grained graphs.

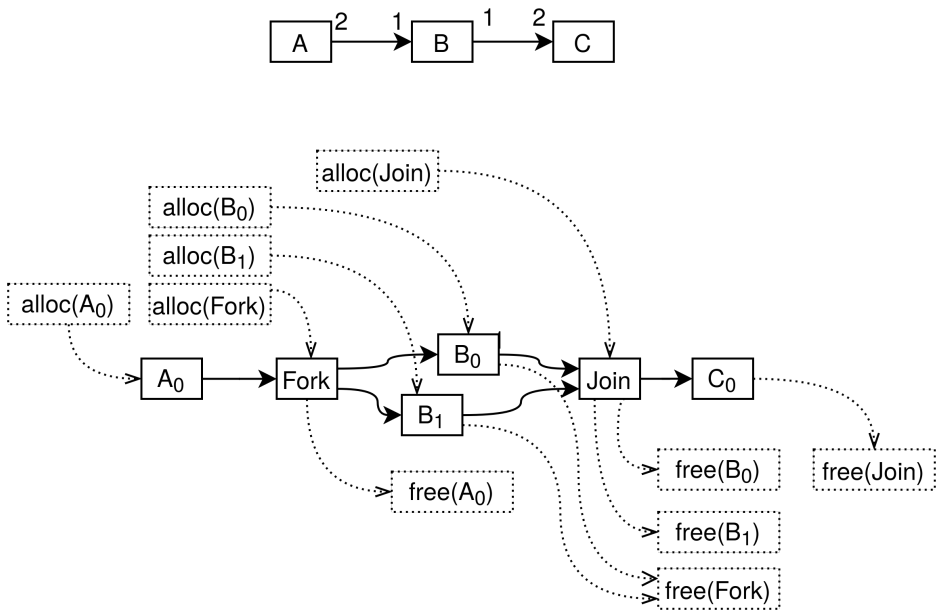

Figure 5: Naïve scheduling of memory management actors for fork and join operations.

If the scheduler is allowed to make assumptions about the memory architecture, it is able to keep track of the mapping between token and memory separately . One such assumption (not present in the original model) is that the output tokens of actors will be *contiguous in memory*. Another assumption is that the actor will be written with reference semantics (that is, the implementation function receives a pointer to the memory containing the tokens). With these assumptions, the offset in memory where the tokens consumed and produced by B_1 are located can be calculated, and the Fork and Join actors become no-ops (Fig. 6). Here, buffer AB is allocated and populated by A_0; the partial order of these operations is represented by the arrows. Because the scheduler knows that the output tokens of A_0 are contiguous, it can immediately fire B_0 and B_1; the Fork actor, which originally did the work of providing B_1 with a valid memory location to read from, is now unnecessary and does not need to be scheduled and the memory accessed by B_1 can be calculated as an offset from B_0 by the scheduler. Similarly, the scheduler knows that the input of C_0 is contiguous, and B_0 and B_1 may write their outputs directly into it. For this, the allocation of buffer BC must come before the firings of B_0 and B_1; the edges generated by the scheduler ensure the correct order of operations. Symmetrically, the free operations must wait for all values in each buffer to be consumed.

More low-level information about the behavior of actors can lead to further optimizations. If we provide the scheduler with the information that B operates in-place, we may avoid the allocation of the second buffer and their allocations altogether, as the same buffer is reused (Fig. 7). Preesm implements this in the form of an annotation system [20], provided separately from the graph topology.

Because these optimizations require information that is not encoded in either the Dataflow topology or the actor implementations, and there is no standardized format to represent this, dataflow programmers who wish to make use of them become essentially

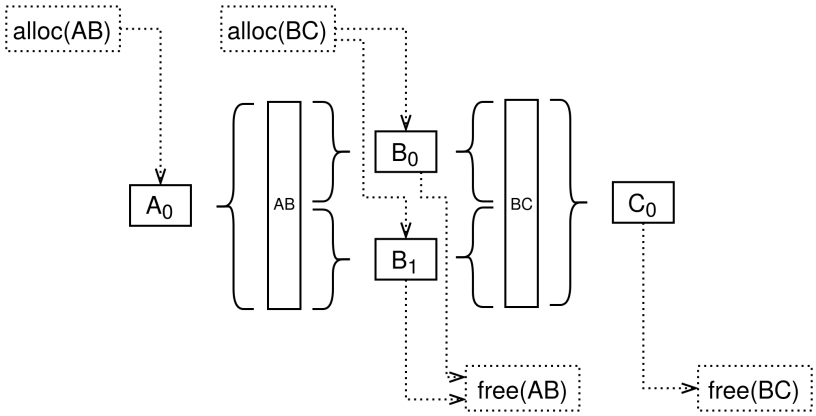

Figure 6: Fork and Join become implicit operations, as the memory addresses passed to B_0 and B_1 can be calculated as views into the AB and BC buffers.

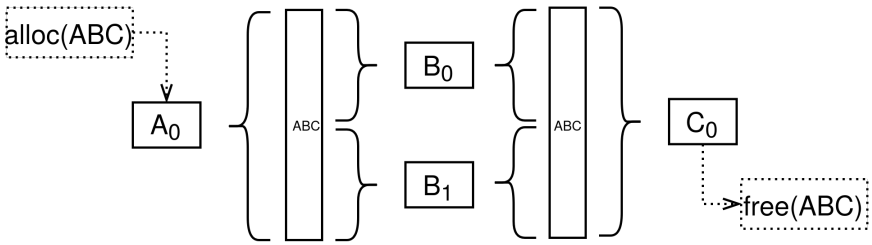

Figure 7: If B writes its output in the same memory as its input, we may reuse the same single buffer ABC and avoid allocations and deallocations.

vendor locked; adapting an algorithm for a different compiler or MoC may require a great deal of manual effort.

In the unpublished article *Memory Allocation in Multidimensional Dataflow exploiting Ownership Semantics* 6.4, we propose a generalization of SDF that encodes the information necessary for these optimizations, as well as support for multidimensional buffers. The article makes use of a novel diagram notation for representing the scheduling of memory operations.

4.6 Hypotheses

With the context presented in this section, we propose the following hypotheses.

Hypothesis 1:

- **Assumptions:**
  - A dataflow graph has some region that can be mapped to an equivalent computation defined in terms of affine loops.
  - Affine loops can be more effectively optimized through traditional compiler workflows than through unstructured single-dimensional dataflow scheduling.

- **Predictions:**
  - A compiler that detects such regions in dataflow graphs and automatically converts them into affine form will present measurable performance benefits, essentially delegating its scheduling to a more specialized algorithm.
  - The performance benefits will be due to improved cache behavior, which will follow from smaller memory footprint and lower synchronization overhead (contribution 6.2).

Hypothesis 2:

- **Assumptions:**
  - Current state-of-the-art does not provide a well-defined path to bridge the abstraction gap between dataflow and traditional compiler workflows. Without a stable interface, works that might otherwise benefit from each other will remain scattered and largely incompatible.
  - The C language remains the standard choice for high-performance kernel implementation, and its memory model is widely-adopted.
- **Prediction:**
  - An extension to classical dataflow MoCs that contemplates memory semantics will enable lower-level optimizations and also facilitate the reuse of C programs as dataflow actors (contribution 6.4).

5 Methodology

Our adopted research approach is described in this section, including choices in the selection of target architecture, reference works, tooling, and development methodology.

5.1 Target platform

As methods of automatic parallelization, the effectiveness of static dataflow schedulers is closely tied to the architecture of the platform. This includes the number of cores, the cache characteristics, the mode of inter-processor communication and the available synchronization mechanisms.

While neither dataflow MoCs nor MLIR are tied to a particular architecture topology, in this work we focus on general-purpose, multi-core RAM architectures. As we are particularly interested in measuring synchronization overhead as core count increases, we perform our experiments on architectures with large numbers of cores. The main development platform is a laptop containing an Intel Core Ultra 5 125H processor, with 18 total threads. It is an heterogeneous architecture, with 4 “performance” cores with simultaneous multithreading, 8 “efficient” cores and 2 “low-power” cores.

We will also run tests in a 24-core Mac Studio with the Apple M2 Ultra chipset. with 16 performance cores and 8 efficiency cores.

5.2 Reference implementations

The primary comparison reference for IaRa is Preesm [55], as it shares the original source models of computation and emits an optimized scheduled in the form of a compiled binary. The main distinction is that, as opposed to a high-level IR like MLIR, Preesm relies on generating C code as a low-level intermediate representation, while the topology parser and scheduler are written in Java. Preesm also does not parse the C source code of actors, lacking information on their internal behavior, while IaRa is capable of parsing both coordination graph and actor code [13].

We also take as reference OpenMP's Taskgraph runtime [79]. While it does not support delays or multi-rates, it serves as a state-of-the-art example of an optimized DAG runtime system.

5.3 Applications

In face of the implementation complexity of large applications in different MoCs, we choose to focus our experiments on two specific applications that present different amounts of task regularity.

5.3.1 Cholesky factorization

This application consists of obtaining triangular matrix $\mathbf{L}$ from hermitian positive-definite matrix $\mathbf{A}$ such that

$$\mathbf{A} = \mathbf{L}\mathbf{L}^*$$

where $\mathbf{L}^*$ is the conjugate transpose of $\mathbf{L}$.

The reference implementation, provided by Taskgraph [79], implements this transformation in the C language using the LAPACK [3] functions, and parallelizes it using OpenMP [17]. Different versions are provided that make use of different feature subsets of OpenMP. Converting the implementation into SDF is challenging, since the parallelization method requires slicing and concatenating of two-dimensional matrices; as such, we consider it a suitable example to demonstrate the benefits of the LADF MoC (section 6.4), in particular when it comes to memory allocation.

The reference implementation relies heavily on nested loops. When profiled, the application displays task regularity; we expect that lowering to affine (section 6.2) and optimizing memory management with high-level multidimensional semantics(section 6.4) will show immediate improvement in performance and memory usage.

5.3.2 Scale Invariant Feature Transform (SIFT)

SIFT is an image processing technique with applications such as object detection and tracking [46]. The implementation provided with Preesm is relatively complex, consisting of 57 nodes and 108 edges distributed through 3 hierarchical layers, also making use of parameters and delays. Because it displays low task regularity, we have selected it as a reference to measure synchronization overhead.

Because of the low task regularity displayed by SIFT [74], we expect that it will not benefit as much from affine lowering as Cholesky factorization. Because it is a large and relatively fine-grained application, we believe it is a useful subject for the measurement of the synchronization overhead of the proposed multi-rate SDF runtime (section 6.1) as

the number of cores grows; its comprehensive utilization of SDF and piSDF features also serves as a useful integration test for IaRa as a whole (section 6.3).

5.4 Profiling and benchmarking

We are interested in several indicators. Wall time, speedup and maximum heap usage is measured using the GNU `time` utility; cache behavior is measured with `valgrind`.

We are also interested in total binary size, and in the relation between size and frequency of allocations.

Per-actor information can be derived using more fine-grained techniques. The `perf` sampling-based profiling tool may track a number of hardware counters and associate them with particular function calls; this information can be used to detect which actors are causing bottlenecks in the schedule. Visual profilers such as Hotspot [37] are able to generate *flame graphs*, a hierarchical representation of resource consumption as it relates to function call chains. Using this visualization, we are able to make qualitative estimates of how task-homogeneous a given implementation is. Some preliminary work has been done on automatically parsing `perf` reports for their potential use in a PGO workflow, to be explored in a future work.

A different approach to profiling is *instrumentation*, which consists of injecting counters and other measurement mechanisms directly inside of the executable. This is a common approach for source-to-source compilers, including Preesm's implementation of the PAPI standard [75], PAPIFY [47]. It is a reliable approach, since it provides an abstracted interface to hardware counters, the usage of which may differ between target platforms.

Instrumentation has an associated overhead in the form of increased binary size, memory utilization and control flow. Since we are interested in making apples-to-apples comparisons with other compilers that may not use the same instrumentation techniques, we opt to focus on the above compiler-agnostic tools.

5.5 Development

IaRa itself is written in the C++ language, with auxiliary scripts written in Python and POSIX shellscript. We follow the development guidelines of the LLVM project and adopt its ecosystem as closely as possible.

The LLVM project has a development culture of fast iteration. Because of the fast evolution of the codebase, interaction with the community is often the best way to acquire up-to-date knowledge. For documentation and support, we rely on the official MLIR website, as well as the LLVM Discourse forum and Discord server.

The LLVM project does not typically enforce stable APIs between different version releases, including MLIR. Because of the non-trivial work related to updating the code to new versions, IaRa was developed against frozen commits of the LLVM and Polygeist projects.

Automated tests are accomplished with the use of LLVM tools LIT [44] and FileCheck [43].

6 Target contributions

The main contribution of this thesis consists of version 2.0 of the IaRa compiler, supporting features that address the limitations covered in section 4. It is a continuation of the author's MsC dissertation [13], where it has enabled a new class of optimization that rely on actor source code analysis [14]. Here, we separate the new compiler features into four contributions, each relating to a separate planned publication. While we cover the features individually, they were developed concurrently and build towards the same global goal: bridging the gap between dataflow and compilers research.

The contributions are:

1. A multi-rate, multi-core SDF scheduler with heap-based memory management that avoids conversion to single-rate dataflow;

2. Automatic conversion of dataflow parallelism into affine loops, and measurement of the impact of integrated affine loop optimizations;

3. The IaRa compiler framework, including the compiler architecture, the MLIR dialect, and an ABI definition for interacting with C actor implementations;

4. The Layout-Aware Dataflow MoC extension that allows for memory optimizations when processing multidimensional data, using MLIR's built-in tensor abstraction.

6.1 Multi-rate SDF scheduler and runtime

State-of-the-art SDF schedulers such as Preesm scheduler rely on converting multi-rate SDF graphs to single-rate form, in order to enable fine-grained ahead-of-time scheduling and allocation [56]. The advantage of this approach is that it reduces the need for runtime synchronization, because the dependencies of individual actor firings is precomputed with a fine-grained scheduling pass. Because IaRa deals with the overhead of fine-grained firings by removing them from the dataflow schedule altogether, the benefit of the conversion to single-rate is diminished. As such, IaRa provides an on-line, task-based synchronization runtime that operates directly with multi-rate semantics, avoiding the drawbacks of the conversion when it comes to binary size and compilation time. While memory management is similar to that of runtimes for dynamic MoCs in that it relies on heap allocation, the predictability of SDF allows for more optimizations.

This contribution is intended to measure the relative usage of dynamic memory utilization in comparison to Preesm, as well as demonstrate the minimum viable product of an MLIR-based multi-core dataflow compiler. A detailed description of the runtime as it exists today follows, as well as thoughts on possible future contributions relating to the algorithm.

The first prototype of the runtime was based on OpenMP Task [53] dependencies; for single-rate graphs without delays such as Cholesky factorization, the graph can be translated into OpenMP primitives without significant change in its topology. While MLIR provides a built-in dialect for OpenMP primitives that makes this conversion straightforward, this approach proves to be insufficient for multi-rate graphs. OpenMP's dynamic task dependency runtime is not capable of registering a dependency drain before registering its source, as the dependency declaration semantics of OpenMP follow the relative order that task dependency directives appear in source code. This is needed for the proposed on-line scheduling algorithm; the number of dependencies for a given firing of each

actor is not constant, as not all tokens being consumed by a firing were necessarily created by the same firing of the producer. For an example, see Fig. 10 in section 6.1.2.

Differently from ahead-of-time schedulers like Preesm which effectively compute and store the order and memory offsets of every firing ahead of time, our approach synchronizes the execution of dependencies and manages their memory through the use of semaphores that are created and destroyed as needed; therefore, OpenMP's task built-in task synchronization can't be used because we cannot know ahead of time that the executions a firing's dependencies will follow a given order.

To address this, we propose a custom dynamic dependency management algorithm that uses a combination of shared-access mutexes, atomic operations and a heap-allocated registry that stores semaphores and memory offsets of actor firings only while they are active, with the goal of reducing memory usage and retaining good cache behavior. The algorithm and data structure are briefly explained in the next paragraph and in Fig. 8.

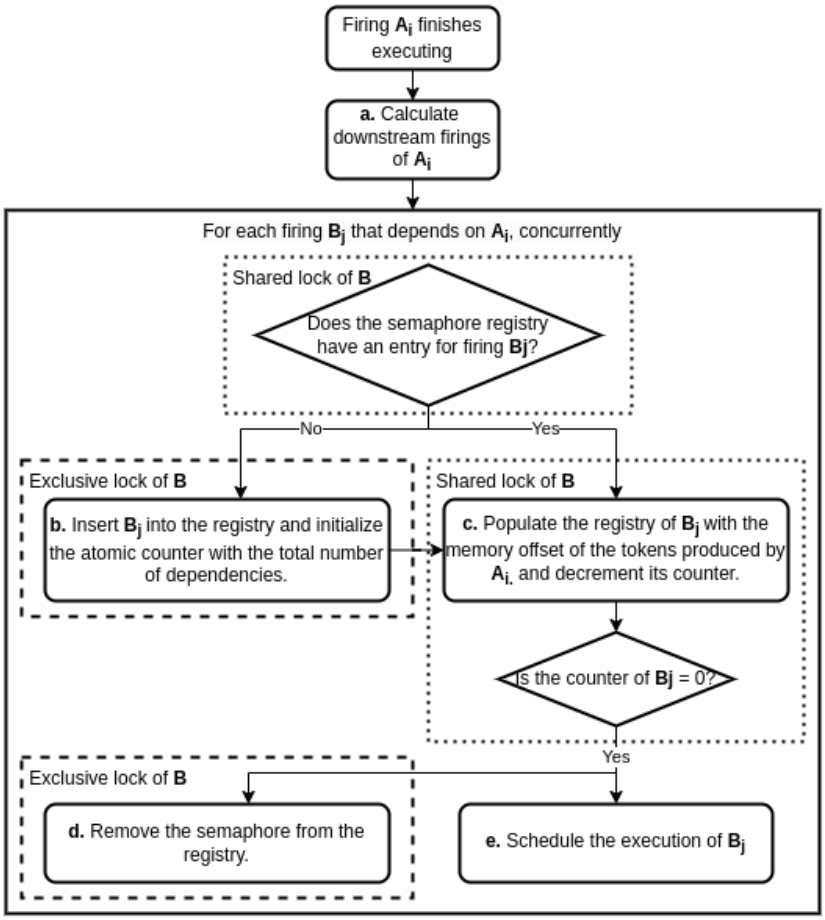

Figure 8: Flowchart describing the on-line scheduling process. Each firing is responsible to schedule the execution of the firings that consume its outputs.

The runtime calculates which firings of which downstream actors depend on the current firing (**a.**). This calculation is inexpensive because it relies exclusively on statically-generated constants. The runtime then checks the semaphor registry for each of the downstream firings, creating an entry if it doesn't exist (**b.**). The information that the dependency was fulfilled is stored by decrementing the associated atomic counter of the downstream firing (**c.**); when the counter reaches zero, all of the upstream dependencies have been executed, the entry can be removed (**d.**) and the downstream firing can be

scheduled (**e.**). To avoid excessive contention, access to the entries of each actor's firings is mediated by a separate shared mutex. Decrementing a counter can be done through a less expensive shared acquisition, while creating and destroying an entry requires exclusive acquisition (only done twice per firing). This is needed for the hash-based data structure, which may cause memory movement on entry insertions and deletions. This mutex and hash-based scheme is at the moment only intended to provide minimally correct behavior while preserving low memory usage, as a way to counteract the fact that affine loop compilation techniques typically require the whole buffer to be allocated at the same time; advanced techniques such as cache-aware and lockless programming [7,25] and concurrent hashing algorithms such as cuccoo hashing [31] can be employed in the future to achieve lower synchronization costs for low task regularity algorithms.

There are potential drawbacks to this approach; while unavoidable with on-line multi-threaded scheduling, mutex and heap operations can be expensive to synchronize. However, following the assumption that much of the synchronization overhead will be offloaded to an affine loop compilation flow, we believe this strategy suffices as a starting point, leaving room for future optimization.

As far as we are aware, this is the first memory allocation scheme that attempts to use static SDF information to improve the performance of heap allocation; most work on heap allocation in dataflow is concerned with unconstrained MoCs such as DPN. While we don't expect that this scheme will beat the low runtime overhead of Preesm's static allocation, we would be interested in a future investigation on the effectiveness of the optimization in comparison to a dynamic FIFO-based version that would be compatible with dynamic MoCs.

6.1.1 Memory allocation and ownership semantics in SDF

In constrast to IaRa 1.0's static memory allocation [13], IaRa 2.0 relies on runtime allocation; while sophisticated strategies for static memory allocation exist for single-rate SDF [20], they have not yet been made compatible with runtime memory allocation with multi-rate semantics.

Because many actor firings may interact with the same memory buffer, some way to represent the dependencies between memory allocation, access and deallocation is necessary. For this, we make use of dataflow semantics themselves. Allocation and deallocation operations are represented as nodes themselves, and all non-read-write ports in the graph are converted into read-write ports (we introduce ports with different ownership semantics in contribution 6.4). Input ports generate a new corresponding output port that flows into the deallocation node of the buffer they read from, and output ports generate corresponding input ports that connect to the corresponding allocation node. The generated edges only represent logical dependency information; the memory information of the allocated buffers is managed directly by the runtime.

6.1.2 Heap allocation, delays and read-write ports

When combining SDF delays (section 2.1.2) with heap-based allocation, we run into the challenge of scheduling allocations while respecting the contiguity constraints of data access. Consider the example in figure 10. If we desire to avoid extra memory copies, actors must write their output tokens directly into the final memory location where they will be read by subsequent actors, and with unaligned reads and writes, this means that each buffer allocation may be rather large. Depending on the values defined for the port

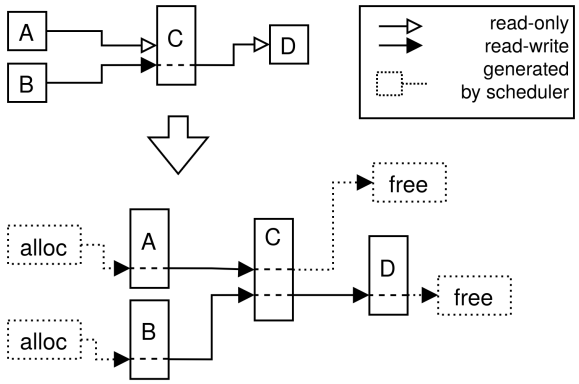

Figure 9: Allocation and deallocation dependencies are represented by actors; all node ports become read-write.

rates and delay lengths, the first allocation must account for the delays, while subsequent allocations do not. This means that the size of the first allocation of the buffer may differ from the subsequent allocations, including the offsets to be accessed by each actor firing. These sizes must be calculated based on the values of the rates and size of the delay.

This is a different approach to the straightforward strategy of copying the input tokens into individually allocated space for each firing when the tokens are unaligned; it may use more memory at a time overall due to the compounding effects of uneven rates, but it trades many small allocations and copies to few, large allocations. It also encourages memory locality.

Each subsequent actor that accesses the buffer (through read-write ports) adds extra constraints to the buffer sizes. This results in a system of linear integer equations of the form

$$D_{ij} + R_i\alpha_i = R_j\alpha_j \forall (i,j) \in E$$
$$R_i\beta_i = R_j\beta_j \forall (i,j) \in E$$

where, for each edge (i, j) in a read-write chain, D_{ij} is the size of its delay and R_i and R_j are the rates of the producing and consuming actors, respectively. We are left with positive integer variables α and β to calculate for each actor, that represent the amount of firings it will execute on that buffer allocation.

It is possible that the system has no solution (for instance, if $D_{ij} \not\equiv 0 \mod GCD(R_i, R_j)$, which happens if the delay causes the inputs and outputs to never tile together. An example is if the rates are equal to 2 and the delay is equal to 1). If a solution exists, the system has exactly one degree of freedom. This can then be algorithmically solved to minimize the maximum allocation size, for instance. If a solution does not exist, the scheduler inserts an extra copy to align the mismatching tokens.

The initial implementation in IaRa solves this system of equations using the mixed integer programming library Pulp [49], with the Coin-OR CBC [15, 65] back-end. To accelerate compilation and avoid the dependency on external tools, a future optimization may leverage the fact that the problem can be described as a system of Diophantine equations to solve it analytically, using a Hermite normal form representation [30].

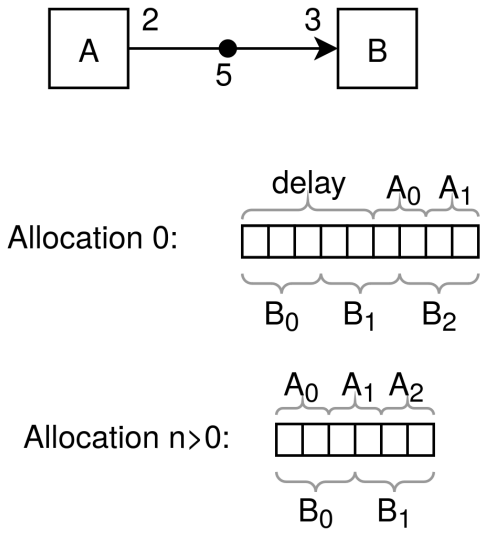

Figure 10: Memory alignment with delays and allocations. The first allocation must contain space for the delay values, while the subsequent allocations do not.

6.2 Affine loop lowering

Consider the following SDF graph and its expansion into individual firings 11 through single-rate transformation. This is the dataflow representation for a program that applies an element-wise transformation to a collection. This is a typical configuration in data processing applications; examples include color space conversion in image processing applications, as well as activation functions in neural networks.

To generate an executable, a dataflow scheduler will typically convert the graph into single-rate form and schedule each firing of actor B independently. Assuming that each individual actor in the expanded graph incurs a constant synchronization overhead, we can expect that the scheduling pressure of the program is proportional to $o(N)$. For schedulers that rely on dynamic synchronization, this can manifest as an extended run time. For static schedulers, this manifests as a longer compilation times and increased binary size.

```
Token* buffer1 = (Token*) malloc(N * sizeof(Token));
Token* buffer2 = (Token*) malloc(N * sizeof(Token));
A(buffer1); // populates buffer with output of A
for (int i = 0; i < N; i++){
    // Read from buffer 1 and write into buffer 2
    B(&buffer1[i],&buffer2[i]);
}
C(buffer2); // use buffer 2 as input of C
```

Listing 1: C implementation of operation in Fig. 11

Listing 1 shows what an implementation of the same application in C might look like. Notice that it is lower-level than the dataflow version: differently from dataflow, it assumes that each element has its own memory address, that they are contiguous and in order, and that the entire collection is held in memory simultaneously [1]

[1] In systems with control over virtual memory mapping, this is not strictly true, as memory ad-

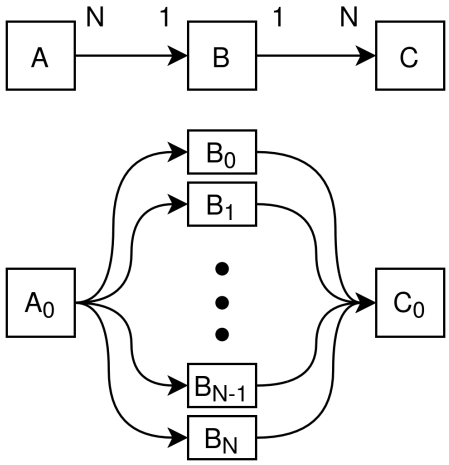

Figure 11: Representation of an element-wise transformation within SDF semantics

Parallelizing this program, even in an automatic fashion, is straightforward. Because we know memory is contiguous, we may rely on pointer arithmetic to separate it into independent memory regions and assign each region to an independent core; the synchronization overhead is proportional to the number of cores. The memory contiguity also incurs the performance benefit of improved cache behavior.

To benefit from this, dataflow programmers must manually create separate graph topologies for the same algorithm, optimized for architecture details such as core count and cache size/amount. While this strategy is successful in preventing the disproportional overhead of overly-granular dataflow graphs, it requires considerable manual effort and hides the inherent symmetry of the graph.

To harness this optimization even in architecture-agnostic code, IaRa provides an optimization pass that automatically detects symmetric fork/join constructs and merges them into an affine loop, which is subsequently parallelized using OpenMP. This also clears the way for more advanced optimization opportunities, such as polyhedral compilation.

6.3 IaRa 2.0

This contribution consists of the new version of the IaRa compiler itself, its MLIR dialect, and a proposal for C ABI for dataflow that retains compatibility with existing implementations.

The IaRa compiler processes a dataflow topology, represented with a custom MLIR dialect, and converts it into standard MLIR that interfaces with the SDF runtime (described in section 6.1). It also generates a header file containing static information about the graph, which is used by the runtime for on-line dependency resolution.

The topology MLIR is lowered to LLVM IR using the `mlir-translate` tool, and the application executable is created by using Clang to compile and statically link the LLVM IR of the topology, the constants header file, the C++ runtime and the C actor implementations.

dresses can be mapped directly into hardware interfaces; however, this optimization is operating-system-dependent and outside the scope of this work.

6.3.1 The `iara` MLIR dialect

Our custom MLIR dialect allows the definition of PiSDF graphs [21], including delays, parameters and hierarchy. The dialect leverages MLIR's native support for hierarchical graph structures to encode actor topology.

In this subsection, we refer to MLIR concepts such as operations, regions, types, etc. Precise definitions can be found in the MLIR language reference [59].

The dialect defines the **`actor`**, **`node`**, **`edge`**, **`in`** and **`out`** operations. Their semantics are explained below, and an example is shown in Listing 2.

The `actor` operation describes the implementation of a (possibly hierarchical) actor. It encodes the signature of an actor, including its name, parameters, read-only input ports, read-write input ports and output ports. PiSDF parameters are represented by the operation's arguments, while ports are represented by `in` and `out` operations in the body of the actor, in order of appearance. The types consumed and produced by the port operations are tensors whose dimension may depend on the values of the parameters, in which case they will be resolved through constant propagation at compile time before code generation.

If the actor body contains `node` operations, it is defined to be a hierarchical graph; its ports act as interfaces to the outside and can be connected to the inner topology through `edge` operations. Conversely, if the actor body does not contain `node` operations, it acts as a declaration; `node` operations in the same module may refer to the actor as their implementation, which will generate a C ABI function call of the same name.

`node` operations represent individual instances of an actor in a topology. Here, they are distinguished from actors for the fact that an actor implementation may be used by more than one node, possibly with different parameters.

The parameters and ports of a node are represented by their operators and results, and the port rates are represented by the dimensions of their tensor types. If an edge of the graph is single-rate, a node input may directly consume the value that corresponds to another node's output, as the data type is the same. Read-write (`inout`) input ports have matching output ports, which must appear in the same order and precede any write-only output ports.

To represent edges that are multi-rate or that have delays, we define the **`edge`** operation. Because the type system of MLIR disallows mismatched operand types, we use this operation as a means to connect node outputs to node inputs. Its single argument and result may be tensors of different dimension, but same element type; additionally, it may define a delay.

```
iara.actor @main  {
  %0 = iara.node @a out: tensor<1xf32>
  %1 = iara.node @b out: tensor<1xf32>
  %2 = iara.edge %0 : tensor<1xf32> -> tensor<1xf32>
  %3 = iara.edge %1 : tensor<1xf32> -> tensor<1xf32>
  %4 = iara.node @c in %2: tensor<1xf32> inout %3: tensor<1xf32> out :
    tensor<1xf32>
  %5 = iara.edge %4 : tensor<1xf32> -> tensor<1xf32>
  iara.node @d in %5 : tensor<1xf32>
} { }
```

Listing 2: IaRa dialect for graph in Fig. 9

6.3.2 Actor ABI

According to the established need for a C-compatible ABI for non-hierarchical actor implementations, IaRa implements the following proposal.

- The name of the actor must match the name of its C (or C-compatible) implementation.
- All functions return `void`. The function arguments consist of the actor's parameters, read-only ports, read-write ports and write-only ports, in this order.
- Ports are represented with bare pointers.
- Using `restrict` qualifiers for pointers and `const` qualifiers for parameters and input ports is optional, but recommended.

Function signatures for the actors in listing 2 are shown in listing 3:

```
void a(float*);
void b(float*);
void c(const float*, float*);
void d(const float*);
```

Listing 3: Function signatures following the proposed C ABI

Note that `c()` has only two arguments; the read-write input and output refer to the same memory, and thus share the same pointer.

Because C function signatures cannot describe multidimensional data [2], the dimensionality of the edges must be described by an actor signature declaration in the topology source code. This actor signature will generate a function call of the corresponding symbol name and signature, allowing kernels to be compiled separately and linked in a separate step.

This is a similar convention to that adopted by Preesm; a contrast is that Preesm relies on manually defining the mappings between dataflow and C through its Eclipse-based IDE; by contrast, our convention relies on embedding semantics into the relative order of C function parameters (mirroring the idiomatic approach in MLIR).

The main purpose of this ABI is to ease the porting of programs into a dataflow-compatible form; it is possible to connect a new kernel into the compiler just by its function signature, avoiding the need to address language-specific semantics from the topology side and vice-versa.

6.4 Layout-Aware Dataflow

As the manipulation of multidimensional data is one of the main focuses of MLIR, it provides extensive support for the manipulation of tensors, both for high-level value semantics and low-level memory mapped semantics. References to memory take the form

[2] While MLIR does define an ABI to represent multidimensional memory references (`memrefs`) using C `structs`, this convention is not yet widely adopted, and most existing high-performance functions in state-of-the-art libraries expect bare pointers to the beginning of buffers. In an effort to ease the writing of "glue code", we adopt the traditional bare pointer call convention as the default; the new convention is nevertheless still available to be generated through a compiler flag.

of `memref` values. Unlike the simple pointers of LLVM IR, `memrefs` carry all the information required for indexing a tensor in memory, up to including affine expressions to convert coordinates between different bases. They also retain alignment and allocation information.

In order to map these features into the semantics of dataflow, as well as to address the issues described in section 4 supporting ownership semantics and an interface to specify memory access, a new, lower-level MoC is required. In the unpublished article *Memory Allocation in Multidimensional Dataflow exploring Ownership Semantics* (available in the appendix), we propose such a model.

The article also introduces *bufferflow*, a novel diagram notation that encodes the relationship between dataflow schedules and memory access.

Once the semantics and multidimensional scheduling capabilities are implemented, a new version of this publication with updated results will be submitted for publication.

7 Chronogram

Table 1 is a chronogram describing the activities executed and planned for the thesis, in three-month increments. The PhD thesis is being conducted as a cotutelle between Brazil and France, represented in the table by white and gray, respectively.

Considerable time was dedicated to building an understanding of the state-of-the-art in both the fields of dataflow and traditional compilers, and to exploring and filtering feasible research directions.

Some notable events include:

- January 2023: Attended the HIPEAC 2023 conference, in Toulouse, France.
- March 2023: *Representing Dataflow in MLIR*, presentation for the MLIR Open Design Meeting [16], an online conference organized by the MLIR community.
- October 2023: *Representing Dataflow in MLIR: challenges and strategies*, presentation given at the academic meeting *MLIR: The Good, the Bad and the Ugly*, organized by GdR SOC^2 and GPL [18], in Rennes, France.

Development of the compiler is as follows:

- January 2023: Starting work on OpenMP-based multi-rate scheduler.
- March 2023: Start refactor, developing new dialect that better utilizes MLIR's features.
- April 2023: Start porting Cholesky implementation to OpenMP-based scheduler.
- March 2024: Start work on expanding capabilites of dialect to represent delays, parameters and hierarchy.
- April 2024: Decided to replace OpenMP-based scheduler, start looking for alternatives/research opportunities
- November 2024: Ported SIFT to new dialect
- November 2024: Started work on new runtime

For the papers to be submitted in 2025/2026, some considered publication venues are:

1. Multi-rate runtime: SAMOS XXV [66] (April 2025)
2. Affine lowering: SBAC-PAD [67] (July 2025)
3. IaRa compiler: TACO/HIPEAC [73] (November 2025)

Attended classes:

1. Integer Linear Programming (S2 2022)
2. French language (S1 2023)
3. Text processing algorithms (S2 2023)

Teaching assistance:

1. C# parallel programming: S2 2022
2. C++ parallel programming: S2 2023
3. Compilers: S2 2023, S2 2025 (projected)

Activity	2022		2023				2024				2025				2026	
	Q3	Q4	Q1	Q2	Q3	Q4	Q1	Q2	Q3	Q4	Q1	Q2	Q3	Q4	Q1	Q2
Events																
HIPEAC conference			•													
MLIR Open Talk			•													
GdR GDP talk						•										
Development																
OpenMP integration			•	•												
Cholesky implementation				•												
SIFT implementation										•						
IaRa dialect			•	•			•	•								
Multi-rate runtime										•	•					
Research																
Literature review			•	•	•	•	•	•	•	•	•	•				
Profiling techniques									•							
Experiments										•	•	•	•	•	•	
Writing																
Paper 1 (runtime)											•					
Paper 2 (affine lowering)												•				
Paper 3 (IaRa compiler)													•			
Paper 4 (LADF MoC)								•						•		
Qualification/Defense											•				•	•
Classes																
Class attendance	•	•	•	•	•	•										
Teaching	•	•			•	•							•	•		
Medical leave									•							

Table 1: PhD schedule per quarter. Stay in France shown in gray.

8 References

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: A system for large-scale machine learning. In *12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16)*, pages 265–283, 2016.

[2] AMD. The ROCm framework. `https://rocmdocs.amd.com/en/latest/`, 2016. [Online; accessed 31-July-2022].

[3] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, third edition, 1999.

[4] Apple. The Metal framework. `https://developer.apple.com/metal/`, 2014. [Online; accessed 31-July-2022].

[5] Florian Arrestier, Karol Desnos, Eduardo Juarez, and Daniel Menard. Numerical representation of directed acyclic graphs for efficient dataflow embedded resource allocation. *ACM Transactions on Embedded Computing Systems (TECS)*, 18(5s):1–22, 2019.

[6] Junjie Bai, Fang Lu, Ke Zhang, et al. Onnx: Open neural network exchange. `https://github.com/onnx/onnx`, 2019.

[7] Naama Ben-David, Guy E Blelloch, and Yuanhao Wei. Lock-free locks revisited. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 278–293, 2022.

[8] Shuvra S Bhattacharyya, Praveen K Murthy, and Edward A Lee. Synthesis of embedded software from synchronous dataflow specifications. *Journal of VLSI signal processing systems for signal, image and video technology*, 21(2):151–166, 1999.

[9] L Susan Blackford, Antoine Petitet, Roldan Pozo, Karin Remington, R Clint Whaley, James Demmel, Jack Dongarra, Iain Duff, Sven Hammarling, Greg Henry, et al. An updated set of basic linear algebra subprograms (blas). *ACM Transactions on Mathematical Software*, 28(2):135–151, 2002.

[10] Adnan Bouakaz, Jean-Pierre Talpin, and Jan Vitek. Affine data-flow graphs for the synthesis of hard real-time applications. In *2012 12th International Conference on Application of Concurrency to System Design*, pages 183–192. IEEE, 2012.

[11] Pierre Boulet. *Array-OL revisited, multidimensional intensive signal processing specification*. PhD thesis, INRIA, 2007.

[12] Juan Manuel Martinez Caamaño, Aravind Sukumaran-Rajam, Artiom Baloian, Manuel Selva, and Philippe Clauss. Apollo: Automatic speculative polyhedral loop optimizer. In *IMPACT 2017-7th International Workshop on Polyhedral Compilation Techniques*, page 8, 2017.

[13] Pedro Ciambra, Mickaël Dardaillon, Maxime Pelcat, and Hervé Yviquel. Co-optimizing dataflow graphs and actors with mlir. In *2022 IEEE Workshop on Signal Processing Systems (SiPS)*, 2022.

[14] Pedro Ciambra, Mickaël Dardaillon, Maxime Pelcat, and Hervé Yviquel. Co-optimizing dataflow graphs and actors with mlir. In *2022 IEEE Workshop on Signal Processing Systems (SiPS)*, pages 1–6. IEEE, 2022.

[15] COIN-OR. Cbc: Coin-or branch-and-cut solver. GitHub repository, Feb 2025. Accessed: 2025-02-21.

[16] MLIR Community. MLIR Talks and Presentations. Online resource, 2024. Compilation of technical talks, tutorials, and design meetings from 2019-2024 about MLIR compiler infrastructure.

[17] Leonardo Dagum and Ramesh Menon. Openmp: an industry standard api for shared-memory programming. *IEEE computational science and engineering*, 5(1):46–55, 1998.

[18] Steven Derrien, David Novo, Mickaël Dardaillon, Simon Rokicki, and Kevin Martin. MLIR: The Good, the Bad and the Ugly. Joint workshop organized by GdR SOC2 and GdR GPL, oct 2023. IRISA, Rennes, France.

[19] Karol Desnos, Maxime Pelcat, Jean-François Nezan, and Slaheddine Aridhi. Pre-and post-scheduling memory allocation strategies on mpsocs. In *Proceedings of the 2013 Electronic System Level Synthesis Conference (ESLsyn)*, pages 1–6. IEEE, 2013.

[20] Karol Desnos, Maxime Pelcat, Jean-François Nezan, and Slaheddine Aridhi. On memory reuse between inputs and outputs of dataflow actors. *ACM Transactions on Embedded Computing Systems (TECS)*, 15(2):1–25, 2016.

[21] Karol Desnos, Maxime Pelcat, Jean-François Nezan, Shuvra S Bhattacharyya, and Slaheddine Aridhi. Pimm: Parameterized and interfaced dataflow meta-model for mpsocs runtime reconfiguration. In *2013 International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS)*, pages 41–48. IEEE, 2013.

[22] Karol Desnos, Maxime Pelcat, Jean-François Nezan, and Slaheddine Aridhi. Memory Analysis and Optimized Allocation of Dataflow Applications on Shared-Memory MPSoCs. *Journal of Signal Processing Systems*, 80(1):19–37, July 2015.

[23] Pierre-François Dutot, Tchimou N'takpé, Frédéric Suter, and Henri Casanova. Scheduling parallel task graphs on (almost) homogeneous multicluster platforms. *IEEE Transactions on Parallel and Distributed Systems*, 20(7):940–952, 2009.

[24] Johan Eker and Jorn Janneck. Cal language report. Technical report, Tech. Rep. ERL Technical Memo UCB/ERL, 2003.

[25] Keir Fraser. Practical lock-freedom. Technical report, University of Cambridge, Computer Laboratory, 2004.

[26] Abdoulaye Gamatié, Sébastien Le Beux, Éric Piel, Rabie Ben Atitallah, Anne Etien, Philippe Marquet, and Jean-Luc Dekeyser. A model-driven design framework for massively parallel embedded systems. *ACM Transactions on Embedded Computing Systems (TECS)*, 10(4):1–36, 2011.

[27] A. H. Ghamarian, M. C. W. Geilen, S. Stuijk, T. Basten, B. D. Theelen, M. R. Mousavi, A. J. M. Moonen, and M. J. G. Bekooij. Throughput analysis of synchronous data flow graphs. In *Sixth International Conference on Application of Concurrency to System Design (ACSD'06)*, pages 25–36, 2006.

[28] Nicholas Halbwachs, Paul Caspi, Pascal Raymond, and Daniel Pilaud. The synchronous data flow programming language lustre. *Proceedings of the IEEE*, 79(9):1305–1320, 1991.

[29] Mary Hall, Jacqueline Chame, Chun Chen, Jaewook Shin, Gabe Rudy, and Malik Murtaza Khan. Loop transformation recipes for code generation and auto-tuning. In *Languages and Compilers for Parallel Computing: 22nd International Workshop, LCPC 2009, Newark, DE, USA, October 8-10, 2009, Revised Selected Papers 22*, pages 50–64. Springer, 2010.

[30] George Havas, Bohdan S. Majewski, and Keith R. Matthews. Extended GCD and Hermite normal form algorithms via lattice basis reduction. *Experimental Mathematics*, 7(2):125–136, 1998. Foundational work introducing the LLL-based HNF algorithm.

[31] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*, chapter 13. Elsevier, 2021.

[32] Julien Heulot, Maxime Pelcat, Karol Desnos, Jean-Francois Nezan, and Slaheddine Aridhi. Spider: A synchronous parameterized and interfaced dataflow-based rtos for multicore dsps. In *2014 6th European Embedded Design in Education and Research Conference (EDERC)*, pages 167–171. IEEE, 2014.

[33] Alexandre Honorat. *Modeling, Scheduling, Pipelining and Configuration of Synchronous Dataflow Graphs with Throughput Constraints.* PhD thesis, INSA de Rennes, 2020.

[34] ISO/IEC. Programming languages — c++, 2020. Sixth edition.

[35] Ivan R Ivanov, Oleksandr Zinenko, Jens Domke, Toshio Endo, and William S Moses. Retargeting and respecializing gpu workloads for performance portability. In *2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 119–132. IEEE, 2024.

[36] Abhinav Jangda and Uday Bondhugula. An effective fusion and tile size model for optimizing image processing pipelines. *ACM SIGPLAN Notices*, 53(1):261–275, 2018.

[37] KDAB. Hotspot: A gui for linux perf. `https://github.com/KDAB/hotspot`, 2023. Accessed: 2023-10-07.

[38] Joachim Keinert, Christian Haubelt, and Jürgen Teich. Windowed synchronous data flow. *Department of Computer Science*, 12, 2005.

[39] John L Kelly, Carol Lochbaum, and Victor A Vyssotsky. A block diagram compiler. *The Bell System Technical Journal*, 40(3):669–678, 1961.

[40] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: A Compiler Infrastructure for the End of Moore's Law. 2020.

[41] Chris Arthur Lattner. *LLVM: An infrastructure for multi-stage optimization*. PhD thesis, University of Illinois at Urbana-Champaign, 2002.

[42] Edward A. Lee and Thomas Parks. Dataflow process networks. *Proceedings of the IEEE*, 83(5):773–801, 1995.

[43] LLVM Project. LLVM FileCheck Tool. Documentation release date: 2025-01-26.

[44] LLVM Project. LLVM Integrated Tester (lit). Documentation release date: 2024-11-13.

[45] Raquel Lazcano López. *Automatic runtime performance optimization methodology applied to dataflow-based hyperspectral imaging cancer detection algorithms*. PhD thesis, Universidad Politécnica de Madrid, 2020.

[46] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.

[47] Daniel Madronal, Florian Arrestier, Jaime Sancho, Antoine Morvan, Raquel Lazcano, Karol Desnos, Ruben Salvador, Daniel Menard, Eduardo Juarez, and Cesar Sanz. Papify: Automatic instrumentation and monitoring of dynamic dataflow applications based on papi. *IEEE Access*, 7:111801–111812, 2019.

[48] Nicholas D Matsakis and Felix S Klock II. The rust language. In *ACM SIGAda Ada Letters*, volume 34, pages 103–104. ACM, 2014.

[49] Stuart Mitchell, Michael OSullivan, and Iain Dunning. Pulp: a linear programming toolkit for python. *The University of Auckland, Auckland, New Zealand*, 65:25, 2011.

[50] William S Moses, Lorenzo Chelini, Ruizhe Zhao, and Oleksandr Zinenko. Polygeist: Affine c in mlir, 2021.

[51] Praveen K Murthy and Edward A Lee. Multidimensional synchronous dataflow. *IEEE Transactions on Signal Processing*, 50(8):2064–2079, 2002.

[52] John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. Scalable parallel programming with cuda: Is cuda the parallel programming model that application developers have been waiting for? *Queue*, 6(2):40–53, 2008.

[53] OpenMP Application Programming Interface Specification Version 5.2. Technical report, OpenMP Architecture Review Board, 2023.

[54] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[55] Maxime Pelcat, Karol Desnos, Julien Heulot, Clément Guy, Jean-François Nezan, and Slaheddine Aridhi. Preesm: A dataflow-based rapid prototyping framework for simplifying multicore dsp programming. In *2014 6th european embedded design in education and research conference (EDERC)*, pages 36–40. IEEE, 2014.

[56] Maxime Pelcat, Pierrick Menuet, Slaheddine Aridhi, and Jean-François Nezan. Scalable compile-time scheduler for multi-core architectures. In *2009 Design, Automation & Test in Europe Conference & Exhibition*, pages 1552–1555. IEEE, 2009.

[57] Rob Pike. The go programming language. *Talk given at Google's Tech Talks*, 14, 2009.

[58] Louis-Noël Pouchet, Uday Bondhugula, Cédric Bastoul, Albert Cohen, Jagannathan Ramanujam, Ponnuswamy Sadayappan, and Nicolas Vasilache. Loop transformations: convexity, pruning and optimization. *ACM SIGPLAN Notices*, 46(1):549–562, 2011.

[59] LLVM Project. MLIR language reference. `https://mlir.llvm.org/docs/LangRef/`, 2024. Accessed: 2025-02-25.

[60] Omair Rafique and Klaus Schneider. Synthesis of parallel software from heterogeneous dataflow models. *SN Computer Science*, 3(3):249, 2022.

[61] Ophélie Renaud, Dylan Gageot, Karol Desnos, and Jean-François Nezan. Scape: Hw-aware clustering of dataflow actors for tunable scheduling complexity. In *International Workshop on Design and Architecture for Signal and Image Processing*, pages 3–14. Springer, 2023.

[62] Ophélie Renaud, Hugo Miomandre, Karol Desnos, and Jean-François Nezan. Automated level-based clustering of dataflow actors for controlled scheduling complexity. *Journal of Systems Architecture*, 154:103217, 2024.

[63] Nadav Rotem, Jordan Fix, Saleem Abdulrasool, Garret Catron, Summer Deng, Roman Dzhabarov, Nick Gibson, James Hegeman, Meghan Lele, Roman Levenstein, et al. Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*, 2018.

[64] Guillaume Roumage, Selma Azaiez, Cyril Faure, and Stéphane Louise. An extended survey and a comparison framework for dataflow models of computation and communication. *arXiv preprint arXiv:2501.07273*, 2025.

[65] Matthew J Saltzman. Coin-or: an open-source library for optimization. *Programming languages and systems in computational economics and finance*, pages 3–32, 2002.

[66] Proceedings of the 25th international conference on embedded computer systems: Architectures, modeling, and simulation. In *SAMOS XXV*, Samos, Greece, July 2025. SAMOS.

[67] Proceedings of the 37th international symposium on computer architecture and high performance computing (sbac-pad). Bonito, MS, Brazil, October 2025. IEEE/SBC.

[68] Sundararajan Sriram and Shuvra S Bhattacharyya. *Embedded multiprocessors: Scheduling and synchronization.* CRC press, 2018.

[69] Milan Stevanovic. *Advanced C and C++ compiling.* Apress, 2014.

[70] John E Stone, David Gohara, and Guochun Shi. Opencl: A parallel programming standard for heterogeneous computing systems. *Computing in science & engineering*, 12(3):66, 2010.

[71] Sander Stuijk, Marc Geilen, and Twan Basten. Throughput-buffering trade-off exploration for cyclo-static and synchronous dataflow graphs. *IEEE Transactions on Computers*, 57(10):1331–1345, 2008.

[72] William Robert Sutherland. *The on-line graphical specification of computer procedures.* PhD thesis, Massachusetts Institute of Technology, 1966.

[73] Transactions on architecture and code optimization (taco) / hipeac 2026. In *Proceedings of the 21st International Conference on High Performance, Edge, and Cloud Computing (HiPEAC)*, TBD, Europe, January 2026. ACM and HiPEAC. Journal-first model through ACM TACO.

[74] The Preesm team. `preesm-apps`. GitHub repository, Feb 2025. Accessed: 2025-02-21.

[75] Dan Terpstra, Heike Jagode, Haihang You, and Jack Dongarra. Collecting performance data with papi-c. In *Tools for High Performance Computing 2009: Proceedings of the 3rd International Workshop on Parallel Tools for High Performance Computing, September 2009, ZIH, Dresden*, pages 157–173. Springer, 2010.

[76] William Thies, Michal Karczmarek, and Saman Amarasinghe. Streamit: A language for streaming applications. In *International Conference on Compiler Construction*, pages 179–196. Springer, 2002.

[77] Hai Nam Tran, Alexandre Honorat, Shuvra S Bhattacharyya, Jean-Pierre Talpin, Thierry Gautier, and Loïc Besnard. A framework for fixed priority periodic scheduling synthesis from synchronous data-flow graphs. In *International Conference on Embedded Computer Systems*, pages 259–271. Springer, 2021.

[78] Sven Verdoolaege. Polyhedral process networks. *Handbook of Signal Processing Systems*, pages 1335–1375, 2013.

[79] Chenle Yu, Sara Royuela, and Eduardo Quinones. Taskgraph: A low contention openmp tasking framework. *IEEE Transactions on Parallel and Distributed Systems*, 34(8):2325–2336, 2023.

9 Appendix

The following pages consist of an unpublished article describing the Layout Aware Dataflow MoC, and the *bufferflow* diagram notation.

Memory Allocation in Multidimensional Dataflow exploiting Ownership Semantics

Pedro Ciambra[1*†], Mickaël Dardaillon[†], Maxime Pelcat[†] and Hervé Yviquel[*]
[*]Instituto de Computação, Universidade Estadual de Campinas (UNICAMP), Campinas, Brazil
Email: pedro.ciambra@ic.unicamp.br, hyviquel@unicamp.br
[†]INSA Rennes, IETR, UMR 6164, Rennes, France
Email: firstname.lastname@insa-rennes.fr

***Abstract*—Dataflow computational models have gained renewed interest due to the rise of data-centric machine learning algorithms. However, these models often struggle to effectively represent information about the storage and exchange of data in memory, especially across multiple dimensions. Such representation is critical for creating optimized schedules and memory mappings, particularly for cache-enabled von Neumann architectures. This paper introduces LADF, an enhanced dataflow model that incorporates multidimensional buffers and ownership semantics as core elements, and bufferflow, a diagram notation for visualizing memory allocation in dataflow. We explore these concepts through a practical example, illustrating their significance in optimizing computational processes.**

***Index Terms*—dataflow, compilation, multidimensional, ownership semantics, bufferflow, ladf.**

I. Introduction

Procedural programming is a familiar paradigm for most developers, characterized by implementing algorithms through a sequence of operations. These operations can read and write to arbitrary regions of a globally accessible memory space and jump to any code region based on the data stored in this memory, aligning closely with the von Neumann architecture prevalent in most general-purpose computers [1]. This paradigm, while flexible and providing reasonably predictable performance, originates from the era of single-core computers. As such, it does not lend itself easily to parallelization. Additionally, procedural programming is challenging to analyze systematically. Arbitrary memory accesses complicate the visibility of crucial factors such as the lifetime of values, memory aliasing, and data dependencies [2]. These elements are critical for the automated generation of accurate and efficient scheduling for parallel algorithms.

Dataflow was created in part to tackle this issue [3]. It reasons about data not by their location in memory, but by the values they carry and the transformations enacted by code. Instead of accessing memory regions, dataflow actors consume and produce data "tokens", which are not associated with any particular memory location at design time. This allows the scheduler to manage the binding of tokens to memory as necessary, enabling performant parallel schedules without special knowledge of the target platform. Dataflow has a wide variety of applications, being useful for high-level synthesis, digital signal processors, distributed systems, and non-conventional computational topologies [4].

However, this high level of abstraction from the memory prevents the foundational dataflow models from being competitive with procedural programming for many applications. In the current landscape of architectures for general-purpose computing, the most successful strategy for the development of high-performance implementations is procedural, parallel cache-aware programming, where knowledge of the target platform can be used to manually structure the memory access of the program in order to minimize cache-related slowdowns.

This requires precise knowledge of memory behavior that cannot be modeled by dataflow, as it has strict separation between topology and kernel implementation. Compiler developers work around this limitation by providing this missing information to the scheduler through other means. Desnos et al. [5] extend the Preesm dataflow compiler [6] with annotations that provide extra information about the behavior of dataflow ports, exposing opportunities for data and memory reuse and allowing kernels to be implemented with algorithms that perform in-place computations (which is the standard for high-performance libraries such as LAPACK [7]).

The ability to expose memory information to dataflow allocators also allows for *slicing*, or the ability to refer to a partial section of a buffer without needing to copy it. While this has been tackled in existing dataflow compilers by the implementation of special behavior for "fork" and "join" actors, they have the limitation of being necessarily memory-contiguous and one-dimensional. Being able to access memory in flexible layouts is essential for data-oriented multidimensional algorithms such as convolutions, which execute the same computation on many sub-regions of few buffers, where data locality matters and copies can be very costly. This phenomenon has led dataflow projects that rely on multidimensional algorithms (such as most modern machine learning frameworks) to delegate the memory-critical access to procedural programming languages, giving up the advantages of a more coarse-grained automated scheduler.

This context leads us to the conclusion that there is a need for a new, lower-level dataflow model that accounts for memory behavior and multidimensional data layout. While nontrivial, these features are not so complex as to completely negate the advantages of dataflow programming, and could enable it for a new class of algorithm.

A. Multidimensional data

Classical dataflow models of computation represent the transfer of data between actors with FIFO semantics; nodes consume tokens from the fronts of input queues and append to the backs of output queues. While this achieves some flexibility when generating a schedule and enables implicit data parallelism, this has the limitation of being strictly one-dimensional and only provides a single way to distribute data between tasks. Any special addressing would require a new actor that produces the data in the expected shape, which adds otherwise unnecessary copies and scheduler pressure.

Conversely, in multi-dimensional algorithms, it is often necessary to split buffers into arbitrarily sized components and along any of the data dimensions, which may result in complex memory access layouts. These partial references can be copied or modified and eventually recombined into the original buffer. A buffer may also have many overlapping references being addressed simultaneously, and the lifetime of its value must be managed to ensure correct data dependency.

While multidimensional references are something that has become common in modern general-purpose programming languages, not many dataflow works explore this type of representation. A notable example is ArrayOL [8], which extends dataflow by allowing the representation of multidimensional data access using linear algebra and the concept of repetition for describing data parallelism.

B. Ownership and borrowing

Memory management continues to be a difficult problem in both industry and academia. One of the main goals being pursued is automated memory reuse: how to detect that a particular value will not be referenced again. This allows the corresponding memory to be reclaimed and reused by whichever memory management solution is being employed by the program. This problem becomes more complex when memory aliasing is introduced; when we allow memory references such as pointers to be transformed, copied and mutated, we must keep track of all other references to the same memory region. We can do this either manually, relying on conventions and documentation to register which section of code is responsible for freeing the memory once used (as in C); delegate it to a "garbage collection" algorithm that will, at runtime, periodically run through the entire memory to find unreferenced values (as in Java or C#); or, more recently, rely on high-level ownership semantic features that have been made available in languages such as Rust [9]. This last strategy has the advantage of lacking both the overhead of a garbage collection algorithm and the requirement of manual programmer intervention, while having the downside of a somewhat constrained programming style that discourages the heavy use of indirection. Nevertheless, the fact that it prevents faulty manual memory management while retaining performance has greatly influenced the rise in popularity of ownership semantics.

One of the hallmarks of an ownership system is type system-based lifetime tracking. Where traditional programming languages allow opaque pointers that can be used regardless of the dependencies of the pointed-to data, a language with ownership semantics tracks variables in terms of the amount, type, and validity of references into its corresponding memory. A variable can be "owned", which means its lifetime is that of the scope it finds itself in and it will be freed once the scope ends; it can be a read-write reference, which does not have responsibility over the memory and will not cause it to be freed; or it can be a read-only reference, which adds the requirement that the function is not allowed to write into the memory in question. To guarantee the correctness of the memory access, the go-to strategy is to enforce *borrow semantics*: an owned variable may be temporarily converted into either a single read-write reference, or one or more read-only references, but not both kinds at the same time. While this adds an extra constraint for the programmer that complicates the implementation of heavily pointer-relying algorithms such as sparse data structures, it simplifies the problem of aliasing to a level that is manageable by an automatic system.

Having the type system track this extra information is also useful from an optimizing compiler's point of view, as it allows optimizations the freedom to insert and remove copies and to move data in memory, while preserving correctness.

Mapping these semantics to the perspective of dataflow models of computation, each token acts as an owned variable that is consumed, and thus its memory freed, as soon as the consuming actor finishes its execution; annotations such as Preesm's memory script allow using read-write references instead of owned variables for in-place actors, and read-only references for broadcasts.

In this work, we propose Layout Aware Dataflow (LADF), a static dataflow model that tackles the issue of multidimensional memory references by using ownership and borrow semantics to manage data allocations and deallocations, allowing for slicing in arbitrary dimensions and avoiding unnecessary copies and memory usage. To illustrate that, we introduce *bufferflow* diagrams that augment dataflow graphs with information on the memory allocation choices performed by the scheduler.

II. Memory management in dataflow

Schedulers for classical dataflow models rely on the assumption that data is written and read only once; a straightforward allocation schedule consists of allocating the memory immediately before the production of the value and frees it immediately after its consumption, while special actors like *fork*, *join* and *broadcast* may be optimized for multiple reads and linear splitting of values. Depending on the specific dataflow model of computation in question, this memory mapping may occur at compile-time or at runtime. This work will focus on runtime memory allocation, but many of the arguments also apply to static allocation schemes based on symbolic execution.

For the purposes of illustration, consider the dataflow graph in Fig. 1. Here, actor Read emits a single token that contains an RGB color point. The Broadcast actor emits three copies of this input token. H, L and S each read one copy and emit the corresponding component of a HSL color point.

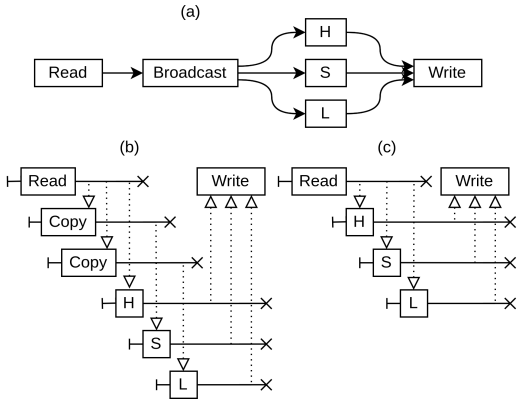

Fig. 1: (a) Dataflow graph with broadcast node. (b) Bufferflow diagram showing memory allocation where the original token is reused once. (c) Bufferflow diagram showing memory allocation with borrow semantics, no copies necessary.

In traditional dataflow scheduling, the broadcast node generates at least two copies, as tokens cannot be divorced from their corresponding memory, and are freed after the execution of the consumer.

To visualize the behavior of this dataflow schedule in regards to memory, we propose the diagram in Fig. 1b, where horizontal lines correspond to a specific buffer in memory, and the left and right end-stops represent their allocation and deallocation in time, respectively. Actors overlapping a buffer alter the contents of that buffer. Vertical dotted lines represent read-only access of a memory region. Here, we see that the broadcast actor generated two allocations. Similarly, H, S and L each generate an allocation in which to store their outputs. Finally, after going through the Write actor, each allocated region is freed. This form of diagram, henceforth referred to as "bufferflow", retains the dependency semantics of dataflow, while adding information about memory allocation and access.

By adding the constraint that actors cannot change the memory assigned to their read-only input ports, we avoid the copies and allocations generated by adding a shared lifetime to the tokens, and by having each node read from the original memory location (Fig. 1c). This means that, rather than simply scheduling the freeing of the memory of each input port after the execution of the actor, we schedule the free only after all reading actors have executed. Since actors may be executing concurrently, this requires synchronization, which, depending on the scheduling and allocation scheme being used, may incur its own overhead. Therefore, we have a trade-off between the overhead of copies plus extra memory consumption and the overhead of synchronization.

While the constraint of read-only input ports is not a big departure from classical dataflow, the inclusion of read-write ports is. While the implementations for classical dataflow actors can be provided in the form of functions where each port maps to a non-overlapping memory region, read-write actors contain one or more pairs of input and output ports that refer to the same underlying memory. This allows the scheduler to prevent unnecessary copies, as long as dependencies are respected.

When a broadcast node flows into multiple read-write ports,

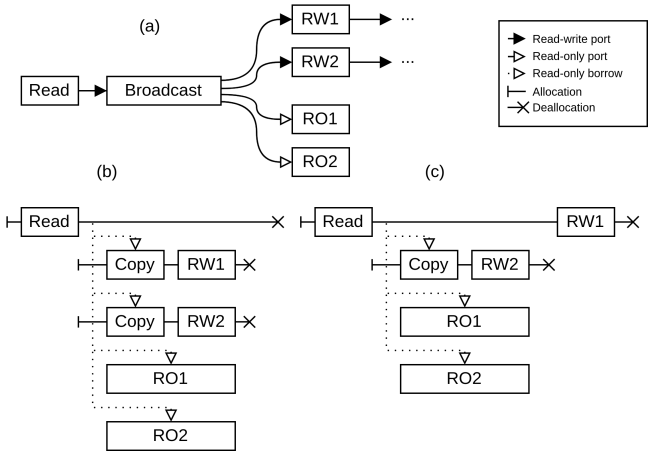

Fig. 2: (a) A broadcast node with two read-only ports and two read-write ports. (b) Scheduling copies for both read-write ports. They can be executed in parallel with the other consuming actors. (c) Reusing original buffer for read-write port RW1. It must execute after the read-only borrows finish.

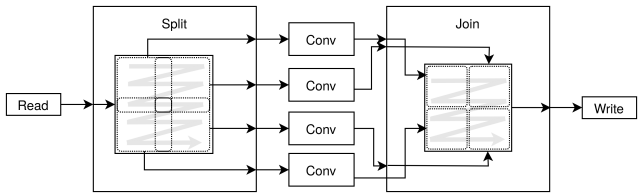

Fig. 3: Parallelization of a convolution. Four overlapping sections of the data must be processed by different instances of the Conv actor, and the results must be joined together into a contiguous image. Memory contiguity is represented by the gray arrows.

extra copies must be generated, transforming read-write ports into read-only ports; the choice of node to which assign the original token is an additional design space decision to be solved by the scheduler. If a broadcast node flows into read-only ports, there is a choice of generating copies, or adding extra logical dependencies to ensure that the read-write actor executes strictly after the read-only actors (Fig. 2).

III. Multidimensional Dataflow

Because traditional dataflow models of computation represent the connections between actors as data streams, the representation of multidimensionality is typically delegated to the implementation of the actors. In the case of 2D data, this consists of a row-major contiguous representation, sometimes parameterized on the dimensions of the matrix. While effective, this hides potentially useful layout information from the dataflow compiler, particularly when it comes to exposing parallelism. Consider the parallelization of a 2D convolution in Fig. 3.

Even though convolutions are highly parallelizable operations, the fact that data dependencies overlap presents a challenge to an automated scheduler. Splitting the input image into multiple parts requires aliasing their overlap, which brings us back to the issue of ownership; the buffer may only be deallocated after all readers have finished executing. Because a traditional dataflow scheduler only has local information about buffer ownership and cannot reason about data dependency

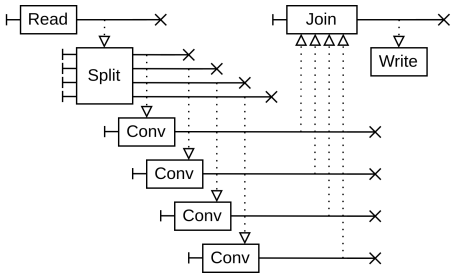

Fig. 4: Memory allocation for Fig. 3 in traditional Dataflow. Each horizontal line corresponds to a separate memory allocation. The Split actor copies each quadrant into a separate contiguous buffer, and Join copies the results of Conv into a new buffer. These actors must be implemented manually.

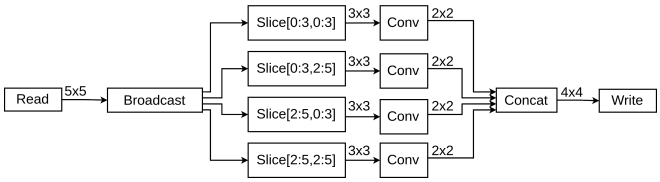

Fig. 5: Convolution of a 5x5 matrix with with Slice actors and a Concat actor in LADF. Slices represented in NumPy notation.

with non-contiguous memory access, as is the case for multi-dimensional slices, this can only be done manually by copying the sections into a contiguous intermediary buffer (in which case the lack of ownership semantics may lead to numerous allocations and copies, as shown in Fig. 4) or with a more fine-grained technique such as polyhedral analysis, which can drastically raise compile times.

With this in mind, we define LADF by introducing into the dataflow model the concept of multidimensional slices: a multi-coordinate view into a buffer that maps into a tensor of arbitrary shape, by way of coordinate transformations. This type of reference is common in numerical processing contexts, being employed by programming languages such as MatLab, R, and Julia, and in frameworks such as NumPy. In the context of dataflow, this has been approached in works such as [8] and [10], but not in the context of memory allocation.

By allowing the type system to carry layout information, we may rewrite the algorithm as Fig. 5. The data layout of the token emitted by the Read actor is encoded as a multidimensional shape; actors that read this token have static access to this layout and can correctly process its contents. Slice actors represent a coordinate transformation and do not generate copies, propagating the read-only reference to their output; similarly, the Concat actor tells the scheduler to allocate the memory in the final configuration and write into it directly, avoiding copies. They are multidimensional generalizations of the Fork and Join dataflow actors, respectively.

This graph generates the schedule in Fig. 6, which performs no extra allocations other than the input and output buffers. The four 3x3 slices, whose extents are represented in NumPy notation, are aliases for the memory of the input buffer. As all of the consuming ports are read-only, the slices may overlap, according to the borrow rules. The dotted lines with solid arrows represent *mutable* borrows, where control over the contents of that section of the output buffer is given to the

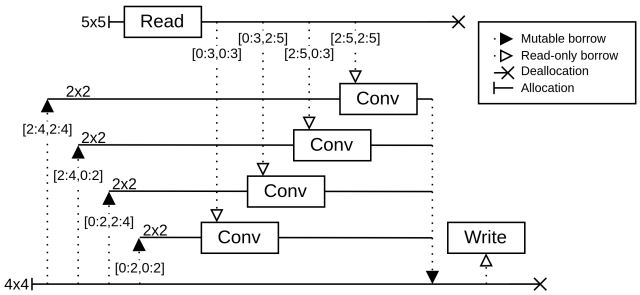

Fig. 6: Memory allocation for the parallelized convolution with slicing and borrowing. Slices represented in NumPy notation.

Conv actor. This is allowed because the four 2x2 slices do not overlap. After the Conv actors execute, write access is "returned" to the buffer, before it is finally read by the Write actor.

IV. Conclusion

In this work we introduce bufferflow, a graphical representation for memory allocation in dataflow graphs. We use it to represent memory allocation in LADF, a new model of computation that raises multidimensional memory information into the dataflow design space. By harnessing ownership semantics, we enable flexible memory optimizations in dataflow that avoid extraneous copies and memory usage when tiling multidimensional algorithms. Future work will explore the application of LADF to automatic tiling in a dataflow context.

References

[1] J. von Neumann, "First draft of a report on the edvac," tech. rep., University of Pennsylvania, 1945. Originally published as a manuscript.

[2] A. Aho, M. Lam, R. Sethi, and J. D. Ullman, "Compilers: Principles, techniques and tools, 2nd edition," 2007.

[3] E. A. Lee, "The problem with threads," Computer, vol. 39, pp. 33–42, 2006.

[4] V. K. Madisetti and I. T. Young, The Digital Signal Processing Handbook-3 Volume Set. CRC press, 2018.

[5] K. Desnos, M. Pelcat, J. F. Nezan, and S. Aridhi, "On Memory Reuse Between Inputs and Outputs of Dataflow Actors," ACM Transactions on Embedded Computing Systems (TECS), vol. 15, p. 30, Feb. 2016. Publisher: ACM.

[6] M. Pelcat, K. Desnos, J. Heulot, C. Guy, J.-F. Nezan, and S. Aridhi, "Preesm: A dataflow-based rapid prototyping framework for simplifying multicore dsp programming," in 2014 6th european embedded design in education and research conference (EDERC), pp. 36–40, IEEE, 2014.

[7] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen, LAPACK Users' Guide. Philadelphia, PA: Society for Industrial and Applied Mathematics, third ed., 1999.

[8] P. Boulet, "Array-OL Revisited, Multidimensional Intensive Signal Processing Specification," tech. rep., 2007.

[9] N. D. Matsakis and F. S. Klock II, "The rust language," in ACM SIGAda Ada Letters, vol. 34, pp. 103–104, ACM, 2014.

[10] P. de Oliveira Castro, S. Louise, and D. Barthou, "A multidimensional array slicing dsl for stream programming," 2010 International Conference on Complex, Intelligent and Software Intensive Systems, pp. 913–918, 2010.